

Developing a Mobile App Using Machine Learning for Garden Flower Recognition

Lakshmipriya Bodhanapu

2201803

A thesis submitted for the degree of Master of Science in Advanced Computer Science

Supervisor: Prof. John Q. Gan
School of Computer Science and Electronic Engineering
University of Essex

August 2023

Abstract

In this era, the machine learning and AI fields explosive growth has had a revolutionary impact across various fields. One intriguing field is image recognition, where machine learning algorithms can precisely recognize and identify objects in photos. As I am very keen to work and know more about image recognition, I intend to develop an Android application for flower recognition as my project. This project focuses on the classification of flowers using deep learning. It also involves developing an Android app that gives users the ability to take pictures of flowers to make predictions in real-time. The difficulty of distinguishing between varieties of flowers based on small distinctions is explored in this study. This is done by using convolution neural networks specifically pre-trained MobileNet and VGG16. The dataset used to train the model is the Oxford 102 flower category dataset. I have achieved 87% accuracy using MobileNet and 78% using the VGG16 model. A TensorFlow lite version of this trained model is then created that is used to integrate with FindYourFlower, the Android application I developed. The user interface of the app is designed in such a way that everyone will find it easy to use. FindYourFlower can classify images with good accuracy and provide relevant flower information.

Acknowledgments

I would like to express my heartfelt gratitude to my supervisor Professor John Q. Gan for guiding me throughout this project, for generously sharing his knowledge, dedicating time for discussion, and for providing continuous feedback that significantly helped to improve my project.

I extend my warmest thanks to my family and friends for providing encouragement and motivating me throughout this project.

Table of Contents

Abstract.....	2
Acknowledgments.....	3
List of Figures.....	5
1. Introduction.....	7
1.1. Project Goals and Objectives	7
1.2 Project Scope	8
2. Literature Review.....	9
3. Dataset source and analysis.....	11
3.1 Planning Model Structure	11
4. Model Development and Evaluation.....	12
4.1 Implemented models	14
A. A Simple CNN	14
B. VGG16	17
C. MobileNet.....	18
4.2 Pre-processing.....	19
4.3 Training and Optimization.....	20
4.4 Model Evaluation.....	21
4.5 Model Conversion.....	25
5. App Development and Evaluation	26
5.1 UI Design and Implementation	26
5.2 Model Integration.....	29
6. Functionality Testing	34
7. User Feedback Analysis.....	38
8. Project Management	41
8.1 Project Plan	41
8.2 Use of Jira	41
8.3 Use of GitLab.....	42
9. Conclusion	43
9.1 Future Enhancement	43
A. Model Improvement.....	43
B. App Improvement	43
Bibliography	44
Appendices.....	47

List of Figures

Figure 1 Datasets stored in Gdrive.....	11
Figure 2 Model code for 5 class.....	12
Figure 3 Model compilation.....	12
Figure 4 Accuracy of 5 class.....	12
Figure 5 Confusion matrix for 5 class.....	13
Figure 6 Model code for 5 class after data augmentation	13
Figure 7 Code for data augmentation.....	14
Figure 8 Test Accuracy.....	14
Figure 9 Code for Loading Datasets	14
Figure 10 Code for splitting dataset.....	14
Figure 11 Rescaling and reshaping	15
Figure 12 Model code for CNN	15
Figure 13 Compilation of CNN	16
Figure 14 Model loss and accuracy.....	16
Figure 15 Train vs Valid accuracy of CNN	16
Figure 16 Model code for VGG16.....	17
Figure 17 Accuracy and loss of VGG16.....	17
Figure 18 Model code for MobileNet	18
Figure 19 Listing images and removing irrelevant files	19
Figure 20 Code of Data Augmentation	20
Figure 21 Code for compiling the Model.....	21
Figure 22 Sunflower prediction.....	21
Figure 23 Wild pansy prediction.....	21
Figure 24 Prediction of wall flower in the web interface.....	22
Figure 25 Prediction of Globe thistle in the web interface	22
Figure 26 Code for producing confusion matrix.....	23
Figure 27 Confusion matrix for MobileNet	24
Figure 28 Train Vs Test Accuracy.....	25
Figure 29 Accuracy vs Loss.....	25
Figure 30 Converting the model into TensorFlow Lite	25
Figure 31 Splash Screen.....	26
Figure 32 Model selection.....	27
Figure 33 Image view	26
Figure 34 Passion flower prediction.....	27
Figure 35 Sunflower prediction	27
Figure 36 Invalid prediction message.....	28
Figure 37 About screen.....	28
Figure 38 Kotlin code for splash screen.....	29
Figure 39 Kotlin code for model selection.....	29
Figure 40 XML code for button creation	30
Figure 41 Kotlin code for the main page	30
Figure 42 Kotlin code for predict function	31
Figure 43 Kotlin code for preprocess image function.....	31
Figure 44 Kotlin code for finding top three prediction	32
Figure 45 Kotlin code for predicting results	33
Figure 46 Firelily prediction using mobile net.....	34
Figure 47 Firelily prediction using vgg16.....	34
Figure 48 Lotus prediction by MobileNet.....	34
Figure 49 Lotus prediction by VGG16	34
Figure 50 Sunflower prediction using VGG16	35
Figure 51 Mobilenet prediction of cauleya spicata.....	35
Figure 52 Pl@ntnet prediction.....	35

Figure 53 Seek by Inaturalist prediction.....	36
Figure 54 Common dandelion prediction	36
Figure 55 Prediction of pink rose.....	37
Figure 56 Prediction of yellow Rose.....	37
Figure 57 Prediction of White Flower	37
Figure 58 Prediction User Feedback.....	38
Figure 59 User feedback on the Interface	39
Figure 60 Performance user feedback.....	39
Figure 61 Feedback on predictions made	40
Figure 62 Overall rating feedback	40
Figure 63 Jira dashboard.....	41
Figure 64 Cumulative flow diagram of Jira	42
Figure 65 Created Vs Resolved Graph	42
Figure 66 Source code directory	47

1. Introduction

In recent years, the rapid advancement of machine learning and artificial intelligence has revolutionized various industries and applications[1]. Image recognition is one promising field where machine learning algorithms have the ability to accurately identify and identify objects within images[2]. Image classification is the process of categorising an image into a set of classes or categories. [3]. In order to accomplish this, a machine learning algorithm must be trained to recognise the visual attributes and characteristics that distinguish different image classes. This aids in identifying differences within similar category of various characteristics. Identifying and labelling specific elements or characteristics of a picture are the main goals of image recognition. [3]. These methods are necessary because contemporary computer vision systems use machine learning methods to efficiently analyse and assess images. One exciting application of this technology is in the field of botany, where a flower recognition app using machine learning can provide a valuable tool for both enthusiasts and experts[4]. In order to recognize flowers via image processing, important characteristics from flower images must first be extracted, and machine learning methods must then be used for classification[5]. So I have decided to choose "Flower Recognition App" as my dissertation because of my keen interest in learning new things. By working on this project, I am enthusiastic about learning and acquiring new skills in the exciting field of android and machine learning.

This project "Develop a Mobile App using Machine Learning for Flower Recognition" aims to develop a user-friendly mobile application that accurately identifies various flower species. Users will be able to take or upload image of flowers by using a simple user interface, and the app will make use of a pre-trained convolutional neural network (CNN) to analyze and categorize the class of the flower. The application will incorporate seamless capability to process images, and accurately predict the flower species. This project gives me the ability to not only follow my interests but also advance my knowledge and skills in these cutting-edge areas.

1.1. Project Goals and Objectives

The main goals and objectives of my project are:

- To obtain a large variety of dataset that contains flower images to classify.
- To train a model by using an algorithm that gives considerably good accuracy to predict flower categories.
- To develop a mobile application that has a user-friendly interface that allows users to take pictures of flowers easily.
- To integrate the developed machine learning model into a mobile application.

1. **Obtaining a Large Variety of Flower Image Dataset:** Obtaining a flower dataset that has a large number of images is essential. The dataset has to be well-curated, with precise labels for each image identifying the type of flower. I found datasets from the pre-existing resources as advised by my supervisor John Gan. The dataset includes the Oxford 102 flower categories and also the flower recognition dataset from the Kaggle which consists of five classes of flower images.

2. **Training a High-Accuracy Classification Model:** The main focus of my project depends on training a model that can classify flower images effectively. In order to classify images, this entails choosing the appropriate algorithm or neural network architecture. This involves using CNN in Python and also I am making use of TensorFlow libraries to train models. I had also advised by my professor about the concept of the pre-trained model resulting in the model to improve accuracy.
3. **Developing a User-Friendly Mobile Application:** Once the model is trained, I should find a platform where everyone can use it. I ended up creating a mobile application that allows the user to take a picture or upload a picture from their mobile phone to see the name of the flower. I decided to use Android Studio for the implementation using Kotlin language which serves as the main language for developing Android applications. This user-friendly app serves as an interface for using the developed model.
4. **Integrating the Machine Learning Model into the Mobile Application:** One crucial step is that the machine learning model that has been trained must be integrated into the mobile application. After completion of the model and app interface. I should integrate the developed model into the app for classifying images. In order to do this, the model must be optimized for mobile devices with respect to both size and computing needs.

1.2 Project Scope

- In this project, my aim is to develop a mobile application capable of identifying various types of flowers by analyzing photos captured using a smartphone camera. This is achieved through the utilization of the Convolutional Neural Network (CNN) algorithm, a powerful tool in machine learning. my primary focus will be on creating a robust deep-learning model that can accurately classify flowers based on their distinctive visual attributes.
- The outcome of this project is an android application as mentioned earlier. Importantly, it is worth noting that the project is entirely self-funded, as the required software tools such as Android Studio, and Google Colab, are all freely available to use.

2. Literature Review

During the planning phase of the project, I have referred various publications and articles in relation to Image classification, CNN and other pre-trained models like MobileNet, VGG, EfficientNet, Resnet. Also for the implementation of mobile application my research went around Andriod Studio for application development.

Research Papers and Articles

Flower classification using CNN:

This research paper explores the application of deep learning techniques like Convolutional Neural Networks (CNNs) and transfer learning for the classification of flowers [5]. The use of CNNs and transfer learning techniques has improved floral classification, according to the authors. On a variety of architectures, including VGG16, MobileNet2, and ResNet50, it assesses the performance of the initial CNN model and transfer learning for tasks involving classification of flowers. Additionally, the paper underlines the practical uses of flower classification, such as monitoring natural habitats, identifying plant species, studying medicinal plants, and supporting the floriculture industry. This paper deals with the same problem and helps in my research to study and experiment with classification using CNN. However, the study states that future work could be the evaluation of different architectures like VGG16, MobileNet2, and Resnet50 to provide insights into the performance and suitability of these models for flower classification tasks [5].

MobileNet:

The research paper showcases the practical results of deploying the MobileNet model using the TensorFlow platform [6]. The objective is to fine-tune it with flower category datasets, prioritizing efficient time and space usage for flower classification tasks. This involves a small trade-off in accuracy when compared to Google's inception-v3 model. The MobileNet model addresses the difficulties in recognizing flower images by effectively cutting down the time and storage demands associated with classification [6]. The paper also highlights the significance of object classification in machine learning and the specific challenges faced in flower classification [6]. It also discusses the potential of the MobileNet model in addressing these challenges and improving the efficiency of flower classification tasks. This article might help me in carrying out my experimentation using MobileNet and examine the results

Deep learning has become increasingly popular, particularly in the area of computer vision, as a result of AlexNet's victory in the 2012 ImageNet Challenge. [7]. Advanced deep learning models like SqueezeNet, SqueezeNext, ResNet, and Inception have taken the role of conventional methods like Canny's algorithm and HOG. This shift also gave rise to new hardware architectures like GPUs and specialized chips that accelerated the training and deployment of these models.

However, the increasing complexity of models posed a challenge in terms of efficiency, particularly for platforms with limited resources. Meeting the demands of real-time applications required models that were both accurate and efficient. In addressing this concern, MobileNet was introduced. It introduced a more lightweight model achieved through the use of Depthwise separable convolutions, which significantly lowered the computational cost. MobileNet also offered hyperparameters like width and resolution multipliers, providing additional flexibility [7]. The key innovation of MobileNet lies in its ability to efficiently handle real-time scenarios while maintaining accuracy.

MobileNet builds compact deep neural networks using an optimised architecture that makes use of depth-wise separable convolutions. [8]. The innovation lies in the introduction of two uncomplicated global hyperparameters that effectively balance the trade-off between processing speed and precision. These hyperparameters empower developers to select an appropriately sized model for their specific application needs, taking into account the limitations of the problem at hand [8].

VGG16:

This study [9] centers on the utilization of the VGG16 model architecture for plant classification. Instead of relying on leaves, the focus shifts to utilizing flower images for classification purposes. This approach is chosen because the shape and color of leaves can exhibit resemblances across various plant species, which might hinder accurate classification.

It introduced VGGnet, which emphasized the significance of network depth in the design of convolutional neural networks (CNNs). This approach presented two variations of the model architecture featuring 16 and 19 layers of weights, respectively. In this current study [9], the author focussed on the VGG16 model, comprising precisely 16 layers of weights. The pre-trained VGG16 structure is made accessible to the broader research community, serving as a valuable resource for future investigations and facilitating transfer learning endeavors.

Transfer learning, a widely employed strategy in deep learning, proves advantageous by mitigating the demand for extensive datasets and computational resources. The VGG16 architecture is particularly well-suited for this technique. To enhance the VGG16 model's performance, it undergoes training with the integration of dropout and data augmentation methods. These strategies have demonstrated effectiveness in counteracting the issue of overfitting commonly observed in neural network models. The primary goal of the research is to combat the issue of overfitting caused by a limited dataset of images [9]. This is achieved by implementing dropout, transfer learning, and data augmentation strategies. The outcomes illustrate that the VGG16 model achieves notably high accuracy rates in classification: 96.25% for the training set, 93.93% for the validation set, and 89.96% for the testing set [9].

In this paper [9], the author dealt with many real-time implementation problems hence it might be hugely beneficial during the experimentation phase

Kotlin:

This research is positioned as a solution to various constraints inherent in Java, and Kotlin's capabilities are put to the test [10]. The study's objective is to empirically evaluate Kotlin's claims in terms of enhancing development efficiency, facilitating maintainability, and simplifying the development process. To achieve this, an experiment is conducted involving 27 teams. These teams are engaged in performing maintenance tasks on Android applications built with either Java or Kotlin. The evaluation encompasses multiple factors, including the quantity of resolved defects, exerted effort, code size, and the participants' perceptions regarding the mitigation of recognized challenges.

The paper strives to furnish substantiated insights into the benefits that Kotlin offers over Java, serving as a foundational reference for future investigations delving into the language's attributes and potential advantages [10]. Kotlin showcased greater conciseness when contrasted with Java, which is evident through a notable contrast in the amount of code produced. During Kotlin development, subjects reported a notably reduced occurrence of NullPointerExceptions. After coming across similar theories I intend to use Kotlin during the Android application development.

Similar Apps

To detect flowers, Many apps, like Pl@ntNet, Picture this, Seek by iNaturalist, plantIN, and many more were accessible through the Google Play Store. [11]. PictureThis is determined to be fairly accurate compared to others. When scanning through a plant, Google Lens also assists in determining the species. The Pl@ntNet is a mobile plant identification app that works without the internet [12]. This advancement enables users to identify plants from any location around the world, even in areas with poor or no network connectivity. This study is encouraging and could greatly aid in the application-building process of my project.

3. Dataset source and analysis

For any model to be trained, we need datasets to experiment with it. Generally, A collection of labeled images is used as a dataset for image classification. The accuracy of a model will always depend on the dataset we are using. For my project, there are many datasets available in Kaggle as well as other resources. As my supervisor suggested I am using flower recognition datasets[13] from Kaggle for getting initial results. It has five categories of flowers namely sunflower, rose, tulip, daisy, and dandelion. This dataset has 4323 images overall. This is an open-source dataset. To train a model,a large dataset of flowers is essential. I also used a large flower dataset which has 102 categories of flowers[14]. This dataset was created by Nilsback and Zisserman. It has flowers which are commonly grown in the United Kingdom. There are a total of 8189 images found in 102 category flower datasets. The authors of this dataset have provided the labels of all images and also their splits. The data splits will have the splits for training, validation, and testing. Since all of these are open sources I am using them for training my model. Figure 1 shows the files of the dataset I am using for my models.

Name	Owner	Last op...	File size	Location
 flowers ★	 me	8 Aug 2023	—	 My Drive
 dataset ★	 me	16:52	—	 My Drive
 imagelabels.mat 👤 ★	 me	25 Jul 2023	502 bytes	 My Drive
 setid.mat 👤 ★	 me	2 Aug 2023	15 KB	 My Drive

Figure 1 Datasets stored in Gdrive

The flower folder will have all 8189 images of flowers from 102 datasets. The imagelabels.mat and setid.mat files are the two files that will have labels and split for the datasets. The dataset folder will have the five classes for flower recognition from Kaggle. I added them to my Google Drive which makes it easier to load the dataset into Google Colab.

3.1 Planning Model Structure

After completing my background research on flower recognition I decided to use the Convolution Neural Algorithm for creating a model. This is because CNN is effective in classifying the images as they are capable of extracting features from the images[15]. It consists of several layers through which they can classify images. Through my research, I found that by using a pre-trained model, we can achieve higher accuracy. Also, this may act as an effective regularizers which can prevent the model from overfitting[16]. Moreover, pre-trained models may also achieve good results on smaller datasets. I am using VGG16 as my base model due to its performance on the ImageNet dataset. VGG16 is very effective in capturing features and it is suitable for complex images [9]. I am planning to use another pre-trained model which is the MobileNet model. I will be integrating the model which gives higher accuracy into my Android app. The advantage of using MobileNet is that it is effective in its specific design for mobile devices which is essential for real-time applications. Based on my research I decided to use dropout layers which will help the model to avoid overfitting due to the large images. I was able to implement all these models in Jupyter initially. Later I started working on collab which was quite faster.

4. Model Development and Evaluation

Once I gathered the required dataset, I started implementing the model using Google Collab. To get initial results my supervisor suggested to use a smaller dataset which is available in the Kaggle. As I mentioned I decided to implement it by using convolution neural networks. The dataset is stored in my drive. The first step is to load the necessary libraries and load the dataset. since my dataset is in Google Drive I should mount Gdrive using Drive.mount(). I set my image's input dimension at 224*224 pixels and batch size at 32. Splitting the dataset into training and testing is the following stage. The data is split in the ratio of 75-25 which means 75% of images will be used for training and 25% will be used for testing. Figure 2 shows how I trained with cnn layers using Keras. First I am recalling my data. This model has three convolution layers of 16, 32, and 64 filters in increasing depth with ReLu activation. To reduce spatial dimensions, I am using max-pooling layers after each convolution layer. Flattened layer ae used to map the feature maps which acts as an input to fully connected layers. I am also using two dense fully connected layers with ReLu activation having 128 neuronsThe number of neurons in the final layer corresponds to the number of classes in the classification task.

```
num_classes = len(class_names)

model = tf.keras.Sequential([
    layers.experimental.preprocessing.Rescaling(1./255),
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes)
])
```

Figure 2 Model code for 5 class

This uses the compile method from TensorFlow's keras[\[17\]](#). I am using Adam Optimizer during training. Adam is a well-liked optimization algorithm that adapts the learning rate throughout training to enhance convergence effectiveness. Sparse categorical cross entropy is the loss function used here that the model will minimize during training. It is also suitable for multi-class classification problems. The model's accuracy will be measured using evaluation measures.

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

Figure 3 Model compilation

```
Test accuracy: 0.599227786064148
```

Figure 4 Accuracy of 5 class

Figure 5 shows the confusion matrix which tells how effectively various types of flowers can be classified using machine learning. So the number displayed in each cell shows how many images were predicted as correct or wrong. The diagonal cell shaded in dark blue resembles the correct prediction of the respective class. Here the class rose and tulips are more confusing. Most of the images of roses are getting identified as tulips and tulips are identified as roses.

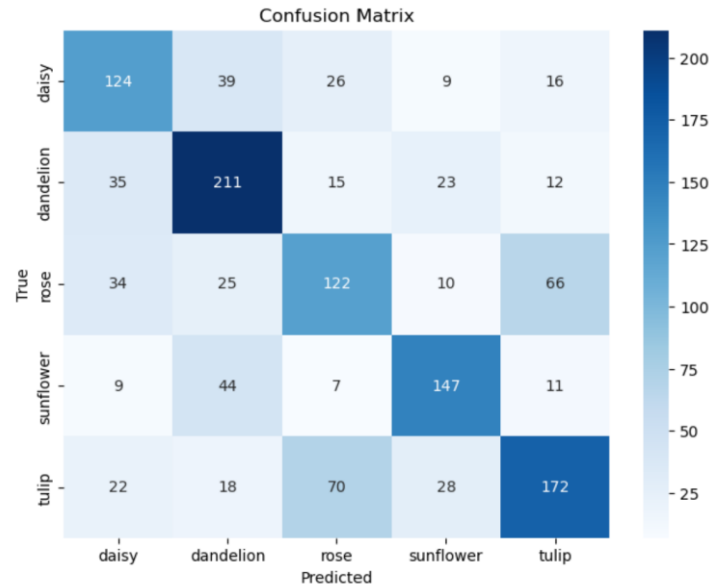


Figure 5 Confusion matrix for 5 class

My model's accuracy after training is 59%. When training for only 5 epochs, accuracy is poor. I set the epoch to 35 to see the performance of my model. This time I am using the dropout layer to my model. Dropout is a regularization technique used to prevent overfitting in neural networks [15]. According to the argument, 0.5, around 50% of the input units will be dropped out throughout training. This reduces the network's reliance on specific neurons, which enhances the generalization of the model. The activation='softmax' indicates that the softmax function will be used as the output of this layer's activation function. The raw output scores of the preceding layer are converted by Softmax into normalised class probabilities. These two layers improve the robustness and accuracy of the model.

```
num_classes = len(class_names)
model = tf.keras.Sequential([
    data_augmentation,
    layers.experimental.preprocessing.Rescaling(1./255),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(128, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
```

Figure 6 Model code for 5 class after data augmentation

Data Augmentation

```
data_augmentation = tf.keras.Sequential([
    layers.experimental.preprocessing.RandomFlip("horizontal"),
    layers.experimental.preprocessing.RandomRotation(0.1),
    layers.experimental.preprocessing.RandomZoom(0.1),
])
```

Figure 7 Code for data augmentation

While implementing this model I also used the concept of data augmentation. This has three sequential augmentation layers. With this, the model is better able to identify objects and patterns in real-world situations by being exposed to a greater range of variables, such as various angles, scales, and lighting conditions. The first layer randomly flips input images horizontally, simulating mirror reflections. The second layer rotates input photos randomly by up to 0.1 radians. To help the model better recognize objects in various orientations, it incorporates slight rotations. In the last layer, the input images are randomly zoomed by up to 10%. It enables the model to manage different object scales and pick up important details. After running for 35 epochs, the accuracy of this CNN model is 77 % which is quite good. Figure 8 shows the final test accuracy.

```
Test accuracy: 0.772885262966156
```

Figure 8 Test Accuracy

4.1 Implemented models

Once I get the initial results with a smaller dataset I have started using the large dataset. I decided to train 102 class large flower datasets using convolution neural networks. I have also tried using pre-trained models like Efficient Net, Mobile Net, and VGG16.

A. A Simple CNN

Firstly, I have started implementing my model using convolution neural networks on the 102-category dataset. I loaded my dataset folder and their labels file. The flower labels are found to be in a Matlab file. To load imagelabels.mat file I am using scipy.io.loadmat() function as in figure 9

```
labelsn = scipy.io.loadmat("imagelabels.mat")
labelsn = labelsn["labels"]
```

Figure 9 Code for Loading Datasets

The dataset was split into two distinct sets for training (trainx and trainy) and validation (valx and valy). These subsets will be used for training. I chose 0.25, which indicates that only 25% of the data would be utilised for training and the remaining 75% will be used for validation. Random_state is an optional parameter that sets the random seed for reproducibility. By specifying a value of 10, the same random splitting will be obtained each time the code is executed.

```
trainx, valx, trainy, valy = train_test_split(train_x, train_y, test_size=0.25, random_state=10)
```

Figure 10 Code for splitting dataset

Preprocessing of my model involves scaling. I have done scaling as it gets pixel values to 0 and 1. The first 2 lines of code in Figure 11 scale the pixel value of training and testing images by dividing them by 255. The training and testing images are reshaped to a weight and height of 150 and 3 color channels assuming RGB images. The reshaping is required to make sure that the data is correctly formatted by the model's expected input shape.

```
trainingimages= trainx/255.0
testingimages=valx/255.0

trainingimages = trainx.reshape((6141, 150, 150, 3))
valx = valx.reshape((2048,150,150,3))
```

Figure 11 Rescaling and reshaping

To implement the model, firstly I initialized a sequential model[20]. I have used three convolution layers of filter 32,64 and 96 in increasing order. The size of the first layer is 5*5 with the ReLu activation function used. It takes image input size of height and width in 150 pixels with 3 color channels. For the other two convolution layers, padding is set to 'Same', maintaining input and output dimensions each of size 3x3, and applying the ReLu activation. There are three max-pooling layers followed by convolution layers for downsampling the data with a 2x2 window and a stride of 2. Flatten() layers flatten the output from the max-pooling layers into a 1D vector to pass it into fully connected layers. I have added a fully connected dense layer with 512 neurons performing ReLu activation. The final fully connected layer has 102 neurons which represent the total classes of the flowers[15]. It also uses soft activation for getting probabilities.

```
model = Sequential()

model.add(Conv2D(filters = 32, kernel_size = (5,5),activation = 'relu', input_shape = (150,150,3)))
model.add(MaxPooling2D(pool_size=(2,2)))

model.add(Conv2D(filters = 64, kernel_size = (3,3),padding = 'Same',activation = 'relu'))
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

model.add(Conv2D(filters =96, kernel_size = (3,3),padding = 'Same',activation = 'relu'))
model.add(MaxPooling2D(pool_size=(2,2), strides=(2,2)))

model.add(Flatten())

model.add(Dense(512,activation='relu'))

model.add(Dense(102, activation = "softmax"))

model.summary()
```

Figure 12 Model code for CNN

To compile my model I use the Adam optimizer which is an effective optimization approach that adjusts the training's learning rate. During training, this is utilised to modify the model's weight. I also tried other optimization techniques but Adam gave good results. Lr=0.001 sets the learning rate to 0.001 which helps in controlling the size of the step while updating weights. The loss parameter describes the loss function that will be used by the model to determine its effectiveness during training[17]. One frequently used loss function is categorical_crossentropy. To evaluate the performance of my model I used metrics parameter which gives accuracy. During training the model's accuracy when tested against training and test data will be monitored and displayed.

```
model.compile(optimizer=Adam(lr=0.001),loss='categorical_crossentropy',metrics=['accuracy'])
```

Figure 13 Compilation of CNN

The model is now ready to get trained on the 102 flower dataset. I set the batch size at 256 for training. This means When the model has processed 256 samples, its weights will be updated. Epoch indicates how many iterations the model will go through on the entire training set. I trained my model for 30 epochs The accuracy of my model is 43% which is too low to predict the images of flowers. There is an overfitting issue in this model. I changed the filters of layers and used the dropout layer at the end to avoid overfitting. Overfitting in CNN arises when the model becomes highly proficient in handling the training data but struggles to extend its predictions effectively to a new, unknown input[15]. After adjusting the learning rate, I trained my model across 40 and 60 epochs. However, the model's accuracy does not change. Based on my supervisor's suggestion I then decided to use the pre-trained model in order to improve the performance of my model.

```
[4.51992904022336, 0.43554688]
```

Figure 14 Model loss and accuracy

Figure 15 displays the evolution of model accuracy on training and validation datasets, showcasing how well the model performs on both the data it was trained on and new, unseen data. Here the accuracy of this model is very low.

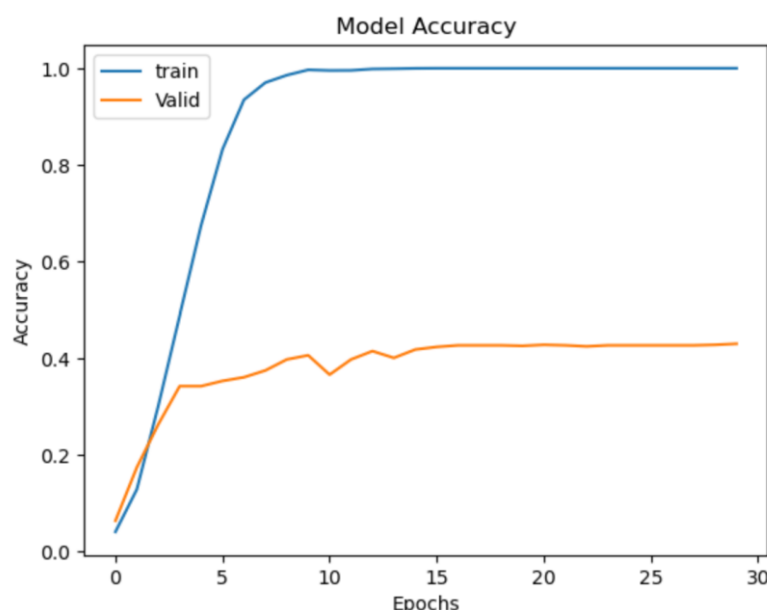


Figure 15 Train vs Valid accuracy of CNN

B. VGG16

Due to the poor accuracy of convolution neural networks, I am using the VGG16 model. The convolutional neural network architecture known as VGG-16, or Visual Geometry Group 16, was developed by academics at the University of Oxford in 2014[22]. This model has a total of 16 layers in its architectural structure. Convolutional and fully linked layers are both included in these layers, each of which serves a specific purpose in the process of classifying images. There are several strong reasons why I decided to use VGG-16 in my project. This pre-trained model has a history of generating outstanding results in many image classification tasks[16]. The idea of transfer learning had a significant impact as I could finetune the architecture of my model using the availability of layers in VGG 16. Compared to other sophisticated models, its basic design enabled greater understanding and evaluation of the characteristics it learned[22]. I created an effective baseline for my project using this model which may help in increasing the performance.

Figure 16 shows the implementation of my model. This model's top classification layers, which are in charge of generating final predictions, were not used. Instead, I tried to modify and finetune the model. I first constructed the VGG16 model with the "include_top=False" argument, omitting the top layers. This input shape was set to (224, 224, 3), which represents images with 224-pixel heights and widths and three red, green, and blue colour channels. By doing this the images are usable for feature extraction and classification in an efficient manner. Using the "layer.trainable = False" line, the layers are iterated through the base model's layers and will become not trainable. In this manner Only the layers I later added will be fine-tuned, leaving the previously trained weights frozen. Further, I added a flattened layer which converts the input from the base model to a flat vector. This will then pass through a dense layer of 256 neurons that is fully linked and activated by ReLu. There is also a dropout layer with a rate of 0.4 to reduce the possibility of overfitting[23]. During training, this layer randomly removes a portion of the neurons, enabling the model to generalize more efficiently. The last layer is the dense layer with 102 neurons specifies how many classes there are overall in the dataset. employing softmax activation. This softmax activation gives the probabilities of this model[15]. I have also experimented with the input shape, dropout out rate, and number of neurons in dense. These final results worked well.

```
base_model = VGG16(include_top=False, input_shape=(224, 224, 3))

for layer in base_model.layers:
    layer.trainable = False

x = Flatten()(base_model.output)
x = Dense(256, activation='relu')(x)
x = Dropout(0.4)(x)
predictions = Dense(102, activation='softmax')(x)
```

Figure 16 Model code for VGG16

After training multiple times with different epoch levels the model is able to give approximately 79% testing accuracy at 53 epochs. the accuracy is 60% and 75% at epochs 20 and 40 respectively.

```
Test Loss: 0.7720, Test Accuracy: 0.7912
```

Figure 17 Accuracy and loss of VGG16

C. MobileNet

To improve the performance of my initial VGG16 model, which had a 79% accuracy rate, I started working with other models. I choose to explore the MobileNet model architecture as an alternative strategy to achieve better results. It excels in a variety of computer vision tasks, such as object identification, and image classification[21]. This model is recognized for its effectiveness in using memory and computational resources[6]. This model is especially designed for mobile applications which might be better as my model will be integrated with mobile applications later. It has a convolution operation named depthwise separable convolution which divides the conventional convolution into two separate layers. A depthwise convolution separates and filters each input channel, and then a pointwise convolution merges the outputs[7]. With the help of the MobileNet architecture, I aimed to enhance accuracy above the 79% accuracy that was attained with my prior model. The implementation of my mobile model is given in Figure 18. The first line creates a base model. I limited my attention to MobileNet's convolutional layers by setting `include_top` to `False` and excluding its final classification layer. The input shape for the model is specified by `input_shape=(224, 224, 3)`. This takes the input of width and height 224. I iterated each layer in the base model using a `for` loop. These layers are frozen so they wouldn't alter throughout training in order to keep things stable and optimize the pre-trained weights. The layers that are added later will be trained. I fine-tuned this model by adding some layers. The base model's output is augmented with the flattened layer. The complex spatial information was transformed into a more manageable 1D format so that the following layers could interact with it more easily. The fully connected dense layers having 256 neurons with ReLu activation are added. In this step, the model uses the flattened data to learn more complex patterns and features. A dropout layer is included to avoid overfitting[19]. The dropout rate is set at 0.4. In order to enable the model to generalize and not become overly dependent on any one particular attribute, this layer randomly eliminates a percentage of the connections on each update[19]. I added another dense layer at the end to get the actual predictions. Since I have 102 distinct classes to choose from, this one contains 102 units. There is a softmax function at the last dense layers which ensures that the output of the model accurately depicts the probability of each class. The file "mobilenet_1_0_224_tf_no_top.h5" was successfully retrieved from the Google Cloud Storage which will be used as the base model of the mobile net[21].

This model is trained at an accuracy of 90%. The test loss of this model is found to be low. my model has a test accuracy of approximately 87%. This states that the model I have developed is giving good predictions over my test dataset. This is a good result and implies that the model is successfully classifying data.

```
base_model = MobileNet(include_top=False, input_shape=(224, 224, 3))

for layer in base_model.layers:
    layer.trainable = False

x = Flatten()(base_model.output)
x = Dense(256, activation='relu')(x)
x = Dropout(0.4)(x)
predictions = Dense(102, activation='softmax')(x)
```

Figure 18 Model code for MobileNet

4.2 Pre-processing

Before pre-processing any data, the necessary libraries should be imported. I started with importing all the libraries which are required for implementation [20]. Os, Numpy, matplotlib, and other crucial libraries were imported. To access the dataset from the drive, google.colab.drive. is used. My deep learning models were built using the 'VGG16' and 'MobileNet' architecture from 'Keras.applications'. Adam optimizer is used for optimization. The evaluation used metrics from "sklearn.metrics," and "EarlyStopping" callbacks were used to track training progress.

Once the libraries are imported, the dataset should be loaded. The flower directory has been loaded through the drive. the imageslabels.mat contains labels for the flower images which is essential to represent which flower belongs to which class as the dataset has 102 categories. On the other hand, I am also using setid.mat file which is used for splitting dividing the dataset into training, validation, and testing sets. These two files are loaded by using the scipy.io function which has access to data in Matlab files. Then the data should be pre-processed which is considered as an initial step. This usually involves removing null values, resizing the image, data transformation, and data augmentation. As a part of pre-processing my data, I am filtering out all the irrelevant items. This includes filtering out the filenames that have ')' as they are not appropriate to the images I am dealing with. Also, I removed the filenames with ipynb_checkpoints which would have been created as a part of the notebook checkpoint. Below figure 19 explains this process.

```
files = os.listdir(image_path)
files = [item for item in files if ')' not in item]
images = [item for item in files if 'ipynb_checkpoints' not in item]
print("Number of images in dataset:",len(images))
```

Figure 19 Listing images and removing irrelevant files

There are 8189 photos in the dataset as a whole.. Before training, I should make sure that all my images are in ascending order as this is going to be mapped with labels. By using function images. sort(), the images in the dataset are arranged numerically. I created three directories in the runtime where the dataset is copied. Using the setid file the images were copied and split according to the ratio into training, validation, and testing folders which are created in runtime. All these images were moved to their respective label folders. The length of the labels is 8189. The total number of images in training, testing, and validation are 6149,1020 and 1020 respectively. For training data, I am establishing an image data generator. Figure 20 will show the implementation. The image dataset will alter the images and enhance the model. The common preprocessing step is to adjust the pixel value between 0 and 1. This is done by using the rescale parameter[18]. The images are set up for rotation to 20 degrees and vertical and horizontal shifts of 20%. horizontal flipping of images is enabled. These enhancements change the original images, allowing my model to accumulate learning from other perspectives. The purpose of the train generator is to load images from the training directory. The images are resized to the width and height of 224 pixels and batch_size is set at 32 to process during training. Since there are multiple classes, the class model is set to categorical. Also, there are two other generators named test_generator and valid_generator which take images from their directory. The images are resized in the standard format of 224*224 pixels. 32 is the set batch size for the test and valid generator. Shuffling of images is disabled to maintain the order of images during testing. This is done by initializing shuffle=false. This will evaluate the model on the testing dataset without changing the image order. Similarly, shuffling is set to false for the validation dataset.

```

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
)

train_generator = train_datagen.flow_from_directory(
    '../content/Training',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    color_mode='rgb'
)

# Validation ImageDataGenerator
valid_datagen = ImageDataGenerator(rescale=1./255)

valid_generator = valid_datagen.flow_from_directory(
    '../content/Validation',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    color_mode='rgb',
    shuffle=False,
)

# Testing ImageDataGenerator
test_datagen = ImageDataGenerator(rescale=1./255)

test_generator = test_datagen.flow_from_directory(
    '../content/Testing',
    target_size=(224, 224),
    batch_size=32,
    class_mode='categorical',
    color_mode='rgb',
    shuffle=False,
)

```

Figure 20 Code of Data augmentation

4.3 Training and Optimization

After creating model architecture using pre-trained VGG16 and MobileNet, the next step is to compile the model. This will make the model to compute more efficiently for training by optimizing the model's architecture[20]. This process usually takes three major components an optimizer, a loss function, and a few metrics to gauge the model's effectiveness. The figure 21 will show the implementation of how I complied with my models. Firstly I am constructing my model by defining the inputs to the base model and outputs to the predictions. This step will create a model that gives a prediction based on the output of the base model. I have chosen Adam Optimizer for my model compilation. After experimenting with

other optimizers like SGD, Adam was working well. My model's weights will be modified by this optimizer during training in order to reduce the loss function. The loss function is the following element. The loss function used here is categorical cross-entropy. This ensures the match between the predicted and actual values[16]. Additionally, the model's accuracy will be calculated to test the performance of the model by using accuracy as a metric. This metric will help me understand how the model works with the training set. When I trained my model initially, the model got over-trained because the validation loss kept on increasing. In order to avoid overfitting my supervisor suggested using early stopping. Here, This method stops model training if the validation loss does not increase for three consecutive epochs. For training, I set to train for 30 epochs and validation data will also be used to check the performance. At epoch 26 the model stops training which means that accuracy is not increasing.

```
model = Model(inputs=base_model.input, outputs=predictions)

model.compile(optimizer=Adam(learning_rate=0.0001), loss='categorical_crossentropy', metrics=['accuracy'])

early_stopping = EarlyStopping(monitor='val_loss',patience=3,verbose=1)

history=model.fit(train_generator, epochs=30, validation_data=valid_generator,callbacks=[early_stopping])
```

Figure 21 Code for compiling the model

After training, the accuracy for mobile net mobile is approximately 87%. Later the list of flower names is labelled as label_names which will have all 102 flower names. Then the flower list is mapped with the index number of labels.

4.4 Model Evaluation

The performance of the model is evaluated once training is done as part of the machine learning[23]. The model's performance is evaluated through many aspects. Since the mobile net model has 87 % accuracy I am testing the accuracy of this model is passing some images as input. Figure 22 and 23 shows the prediction of the image I tested. The model gives correct predictions. From the Figure 22, we can say that my model is predicting correctly as a sunflower even though the images have multiple flowers in them though the trained images are not like that. The figure 23 image is correctly identified as a wild pansy. Another evaluation metric I have used to evaluate the model is the confusion matrix.



Figure 22 Sunflower prediction



Figure 23 Wild pansy prediction

I also created a user-friendly web interface as an additional feature to my project. This is also one of the ways to evaluate my model's prediction before integrating it into a android application. This is done by using the Gradio library [24], which will be used to create web interfaces. This web interface will allow the user to upload a picture of a flower to know their prediction. By clicking on the predict button. My model processes the image and displays the prediction along with the confidence score[24]. The interface improves accessibility by allowing users to interact with the model's predictions. Every time someone runs the model code they can test their prediction through this web interface. The below screenshots show the predictions. The figure 24 shows that my model is predicting wall flower correctly with a confidence of 0.93. The figure 25 shows the correct prediction of globe thistle with a confidence of 0.92. The predictions are correct and the confidence is good.

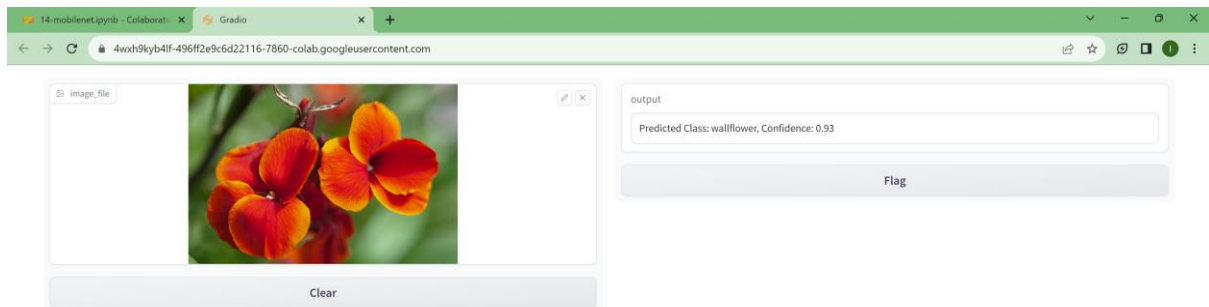


Figure 24 Prediction of wall flower in the web interface

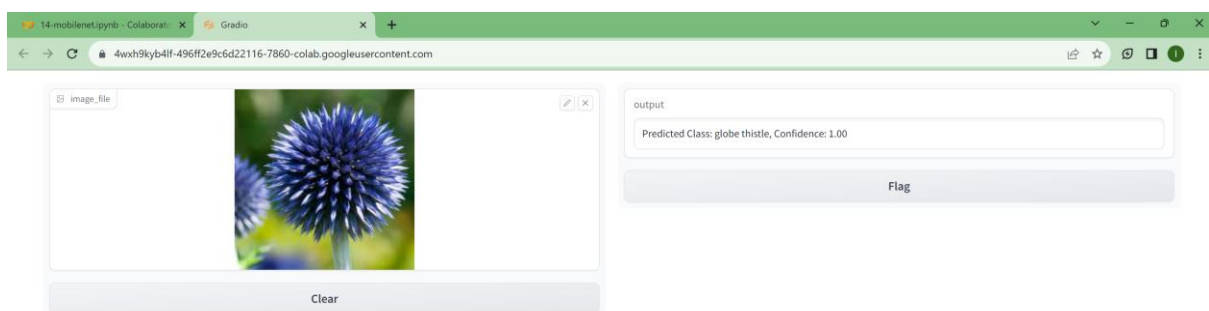


Figure 25 Prediction of Globe thistle in the web interface

The implementation of the confusion matrix is given in figure 26 below this involves Using a validation dataset and the predict function, and the model's predictions are obtained. The arg max function is then used to translate these predictions into predicted labels. The true labels are obtained from the valid dataset. The confusion_matrix function from the sklearn.metrics library is used to build the confusion matrix. In order to display the confusion matrix heatmap with class labels on both axes, the plot_confusion_matrix function is finally used. This is one of the methods for assessing the performance of the my model on different classes.

```

predictions = model.predict(valid_generator)

predicted_labels = np.argmax(predictions, axis=1)

true_labels = valid_generator.classes

def plot_confusion_matrix(confusion_mat, class_names, display_subset=True):
    plt.figure(figsize=(20, 20))
    sns.set(font_scale=0.5)
    sns.heatmap(confusion_mat, annot=True, fmt="d", cmap="Greens", xticklabels=class_names, yticklabels=class_names)
    plt.xlabel('Predicted Labels')
    plt.ylabel('True Labels')
    plt.title('Confusion Matrix')
    plt.xticks(rotation=90, ha="right")
    plt.yticks(rotation=0)
    plt.show()

confusion_mat = confusion_matrix(true_labels, predicted_labels)

plot_confusion_matrix(confusion_mat, flowers_list)

```

Figure 26 Code for producing confusion matrix

The confusion matrix of the mobile net model is given in figure 27. This is used to assess how well the model is doing. It will have true and predicted classes in the x and y axis respectively. The diagonal cells which are in the green colour represent how many images were predicted correctly. The colour complexion represents whether the flower class has more correct predictions or not. If the colour is dark green is the diagonal cell, most of the images in that respective cell is predicted are correct. If the cell is dark green is rest of the cells other than diagonal, then the images are predicted as wrong. Here I have taken 1020 images from my dataset which is from validation dataset. There are around 10 images per each class. This data has been used for producing confusion matrix. Predicted labels are represented on the x axis, and true labels are represented on the y axis. The numbers in the cells represents how many images are predicted as correct or wrong. For example, from the analysis of figure 27, it is seen that the first class pink primrose has 10 images. Among that 5 images are identified correctly and rest of them were classified as different flower. This is due to similarity of pink primrose flower in other classes. My model fails to identify currently. Globe thistle which is second class has 10 images in validation dataset. All of them were predicted correctly. In the third class, blanket flower, 8 images were predicted correctly and remaining 2 images are predicted wrong.

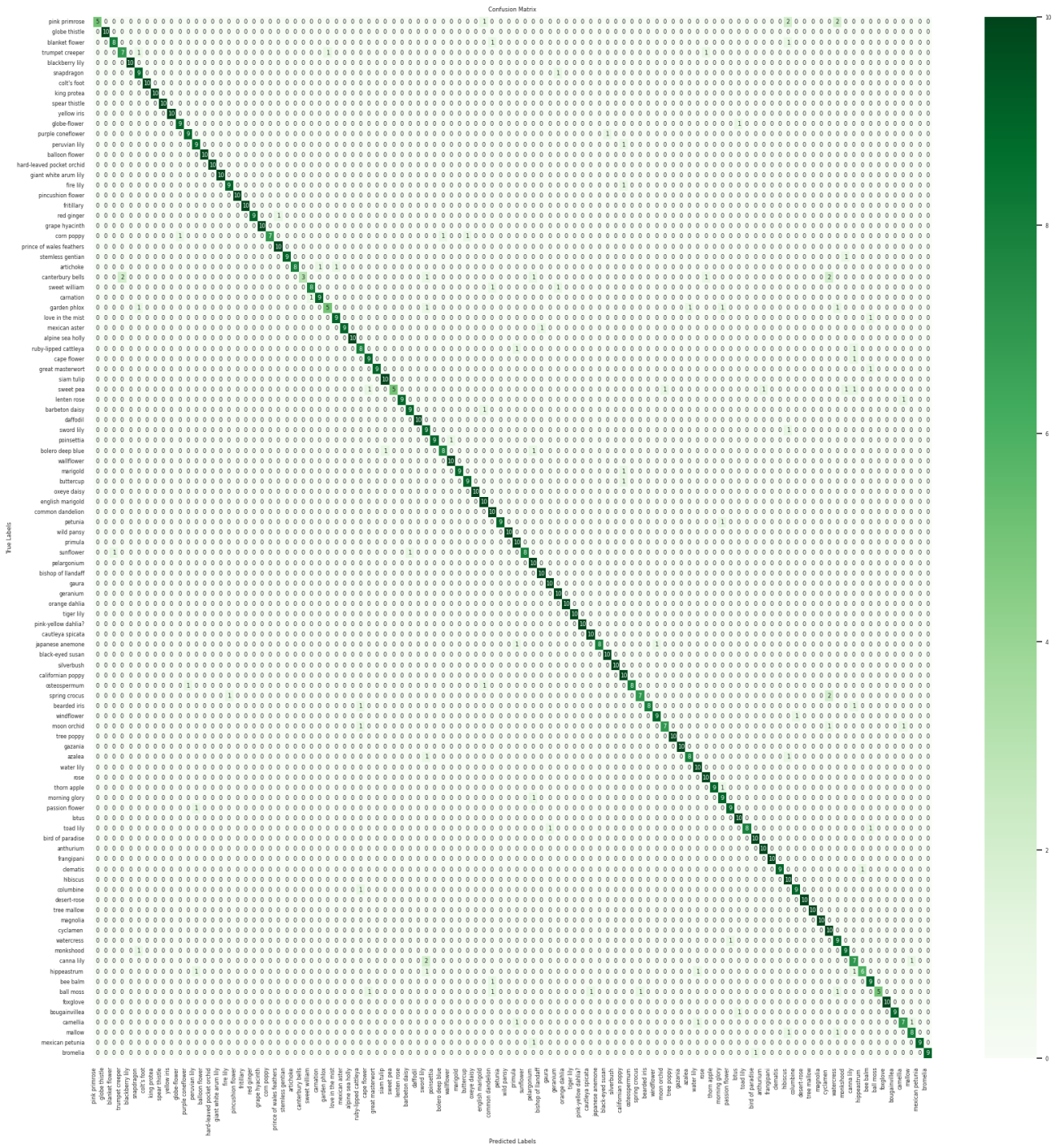


Figure 27 Confusion matrix for MobileNet

Figure 28 shows the representation of training of my mobile net model through plotting. The accuracy of training and testing is shown. It is observed from the graph that my model is facing slight overfitting. The accuracy describes how well my model is responding to unseen data. This can be improved in future work. Figure 29 represents the Accuracy and loss of MobileNnet model. The loss decreased from 4 to 0.4 during the training. The accuracy of this model is 87% approximately.

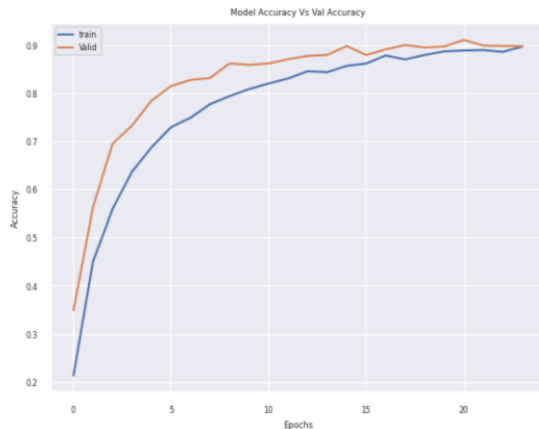


Figure 28 Train Vs Test Accuracy

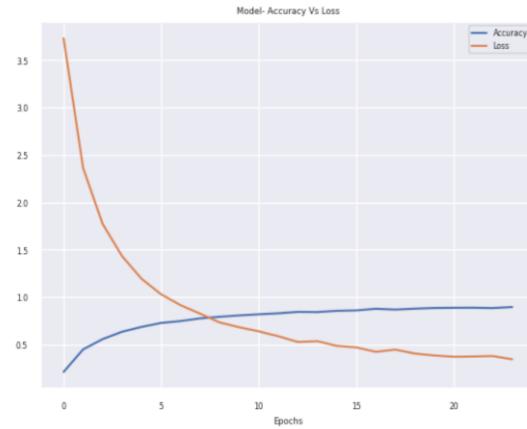


Figure 29 Accuracy vs Loss

4.5 Model Conversion

A neural network learns the configurations and weights required to make predictions on different data as it is trained. After training, we should save the model to use again without running from the beginning. The `save('model.h5')` method saves the model using the h5 extension. By saving the model, we can avoid the necessity of retraining it each time we need to utilize it for making predictions or developing it later.

As the outcome of this project is going to be a mobile application, a model with high prediction needs to be integrated with the app. The model's architecture is loaded using the `load_model` method from the `tf.keras.models` module [27]. This will take the previously saved model. I turned this Keras model into a TensorFlow Lite format that is smaller and better suited for mobile devices. The converted model is saved in binary format. This will write the convert model into `model.tf.lite`. The model can be deployed using this file on mobile devices, embedded systems, or edge devices, ensuring efficient and fast computations using machine learning.

```
import tensorflow as tf

model = tf.keras.models.load_model('/content/my_model.h5')
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()
open("model.tflite", "wb").write(tflite_model)
```

Figure 30 Converting the model into TensorFlow Lite

5. App Development and Evaluation

In the context of developing an application for my project, To make sure that a deep learning model is seamlessly integrated into the software, I strategically chose to use Android Studio as the main development environment [25]. Due to Kotlin's appropriateness for creating Android apps, I chose it as my programming language[26]. While I was initially unfamiliar with Kotlin and how to create Android apps, I was able to use the groundwork I gained through my coursework in mobile and social application programming which helped here. This method was implemented to have a strong and efficient integration of the deep learning capabilities of my model with the accessible user experience of the android app.

5.1 UI Design and Implementation

Before designing the interface of the app, I decided to give a name to represent it. So FindYourFlower, The name of the app that represents its objective, is chosen thoughtfully to encapsulate its essence. In complementing this, a logo that serves as a visual representation is incorporated with the concept of the app. Once the user clicks on the FindYourFlower app the first screen that comes is the splash screen. This splash screen will stay active for 3 seconds. In this splash screen app logo will appear. Also to make the splash screen more attractive, a tagline that explains the app function concisely added. As my project is about the recognition of flowers, A flower discovery companion is the tagline I have added. To make the interface of the app more colourful, a vibrant background image of the flower is included. Adding images directly to the background makes the text invisible due to the mismatching colour used.so I preferred a blurred image to make it more prominent and visually appealing. Figure 31 shows the splash screen of the app.



Figure 31 Splash Screen

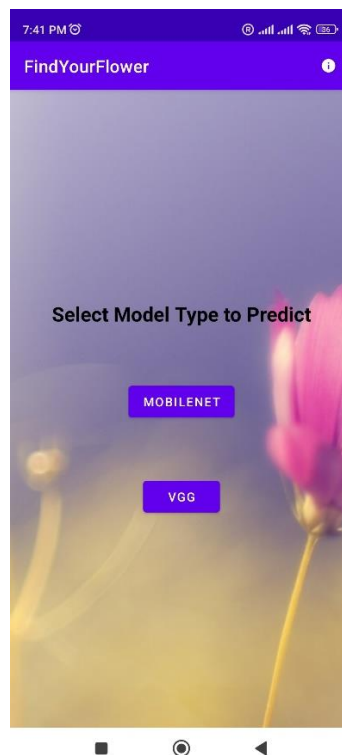


Figure 32 Model selection Screen

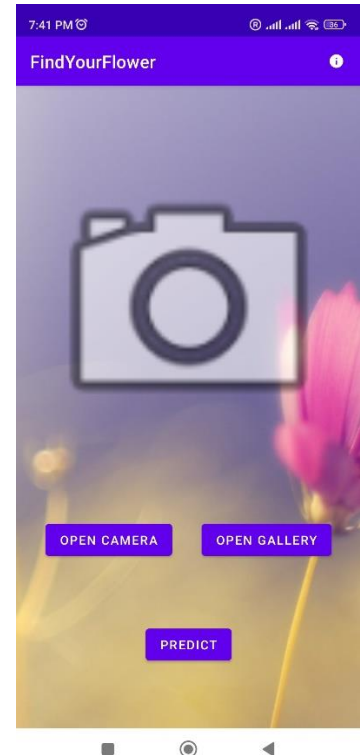


Figure 33 Image view screen

After displaying this screen for 3 seconds it will navigate to the next screens. It consists of two buttons namely MobileNet and VGG as shown in figure 32. These are the two models which I implemented as a part of my project. The user should select any one model to proceed further. Once the user clicks on any one of the models displayed, it will navigate to the next screen. The next screens will have an image view to display the selected image, a button to capture the image directly from the mobile's camera, a button to upload the image from the phone's gallery, and a predict button to predict the flower category with the help of the selected model. Figure 33 shows the design view of the page once the type of model has been selected.

By clicking on the predict button to get the prediction of the uploaded or captured flower image, it will show the name of the flower and the confidence of the predicted flower on another screen. I have given the condition to display the top three predictions above 10% means that the first top three prediction percentages of more than 10% only will be displayed on the prediction results page.

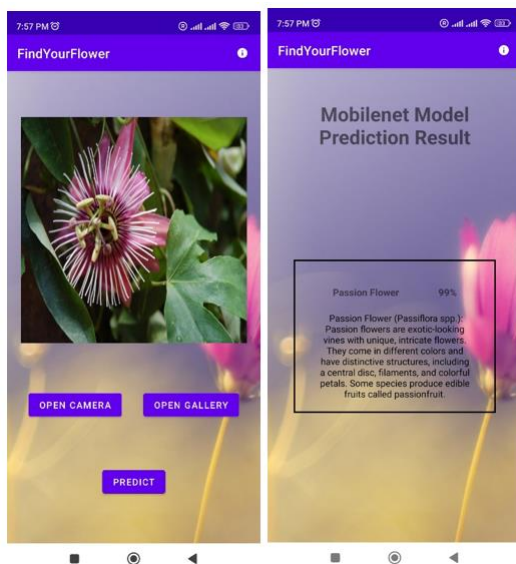


Figure 34 Passion flower prediction

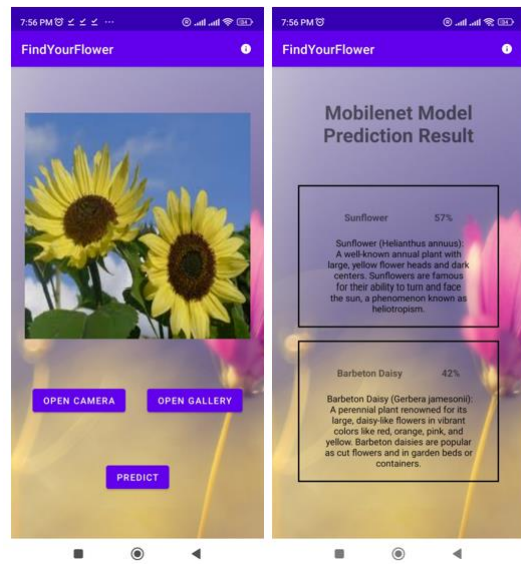


Figure 35 Sunflower prediction

Figure 34 shows the image view of the flower image uploaded, after clicking on Predict on the next screen the results are displayed. it is correctly predicting as passion flower with confidence 99%. Also, the prediction of sunflower flower images is found to be 57% and it is showing the 2nd prediction of 42% which is the Barberson daisy as in figure 35. This is because the Barberson daisy and sunflower look similar. In addition to the flower name and confidence percentage, it will also give a few descriptions of the predicted flower below each prediction. This will be helpful for the user to know more about the flower if they don't have any idea about it.

Also to make the app to show valid predictions on the images I set it to throw an error message for the images whose predictions are less than 20 percent which is an additional feature. figure 36 shows the invalid error message of uploading a random picture that is not flower.

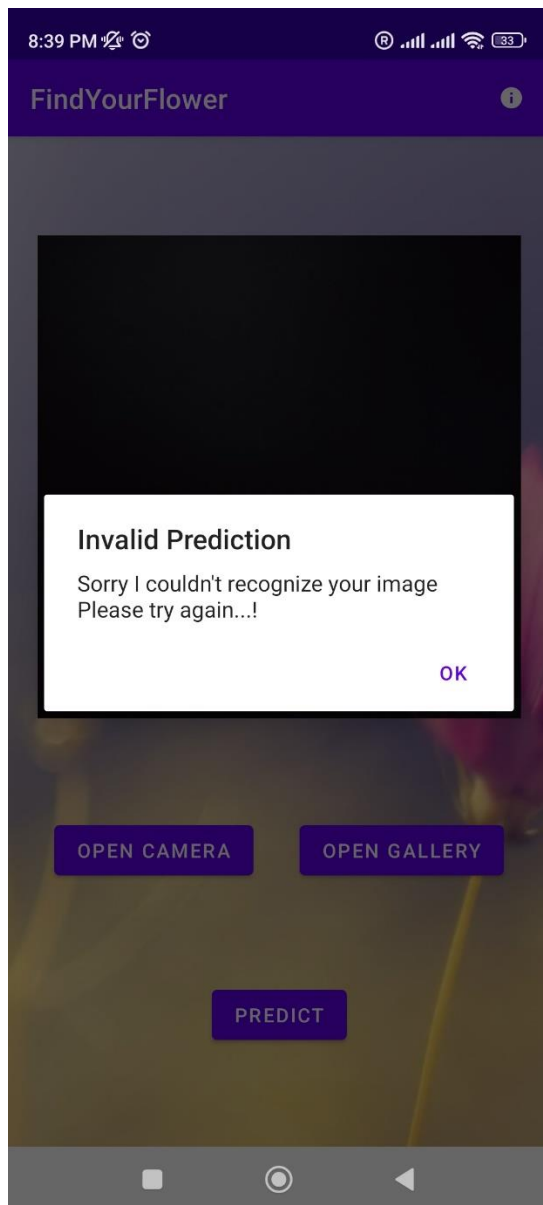


Figure 36 Invalid prediction message

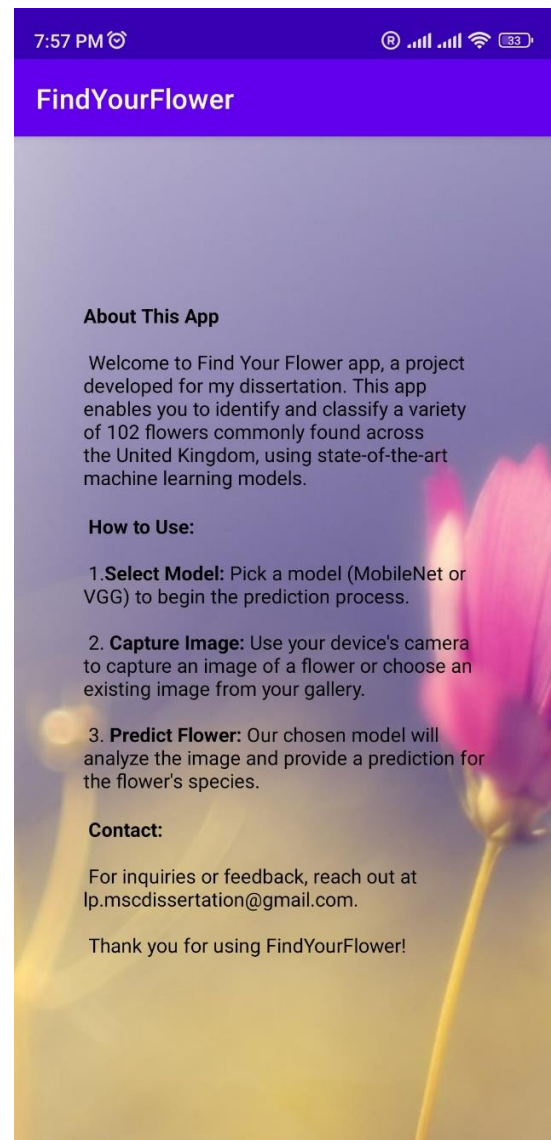


Figure 37 About screen

Also on the top right corner of all screen, there is an info button enabled. This is used to give basic information about the app and how to use it. This is one of the additional features of the app the figure 37 shows what kind of information will be displayed on information page.

5.2 Model Integration

The process of integrating the models within the app and the creation of the app requires the implementation of several key functions[26]. There is an inbuilt option to load the tensor flow lite models from the local machine[27]. I have loaded my two Lite models into Android Studio which I will further use for getting predictions.

The below figure 38 shows the snippet of the main activity which is a splash screen that appears for 3 seconds. It then directs the user to the next screen which is model selection activity where the user selects the model. This is done by using the intent method. On create option menu method is to create a menu to be displayed in the activity toolbar. The "About" menu item initiates an intent to start the "AboutActivity" when it is selected. This approach improves user involvement by enabling them to access more app-related information.

```
class MainActivity : AppCompatActivity() {
    private val SPLASH_DISPLAY_LENGTH = 3000L // 3 seconds
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Handler().postDelayed({
            val intent = Intent( packageContext: this, ModelSelectionActivity::class.java)
            startActivity(intent)
            finish()
        }, SPLASH_DISPLAY_LENGTH)
    }
}
```

Figure 38 Kotlin code for splash screen

Below figure 39 explains how I created buttons in the model selected page. Model selection activity class establishes the model selection functionality within the app. I used the binding technique to set up the UI components. This enables two buttons MobileNet and VGG16 displayed as clickable elements. Each element initiates an intent to open the "FindFlowerActivity" when clicked and passes relevant data, such as the model path and title. With this design, users have an easy way to select a model for classifying flowers. Figure 40 shows the XML code to create a button. The "MOBILENET" text is configured on the button, which is set up to change in width and height depending on the content. It is positioned making use of constraint layout constraints so that it is positioned above an element with the ID "VGG" and below a view with the ID "textView2"[25]. The button's horizontal position spans the entire parent layout, from beginning to finish. It is vertically centered inside the available area because its vertical positioning bias is set at 0.5.

```
class ModelSelectionActivity : AppCompatActivity() {
    lateinit var binding: ActivityModelSelectionBinding
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityModelSelectionBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.mobilenet.setOnClickListener { it: View?
            val intent = Intent( packageContext: this, FindFlowerActivity::class.java)
            intent.putExtra( name: "ModelPath", value: "mobilenet.tflite")
            intent.putExtra( name: "Title", value: "Mobilenet")
            startActivity(intent)
        }

        binding.vgg.setOnClickListener { it: View?
            val intent = Intent( packageContext: this, FindFlowerActivity::class.java)
            intent.putExtra( name: "ModelPath", value: "VGGModel.tflite")
            intent.putExtra( name: "Title", value: "VGG")
            startActivity(intent)
        }
    }
}
```

Figure 39 Kotlin code for model selection


```

<Button
    android:id="@+id/mobilenet"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="MOBILENET"
    app:layout_constraintBottom_toTopOf="@+id/vgg"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/textView2"
    app:layout_constraintVertical_bias="0.5" />

```

Figure 40 XML code for button creation

I initialized multiple important variables when creating the "FindFlowerActivity" to make the app's operation easier as in Figure 41 . One of these variables, "MODEL_PATH," keeps track of the path to the model that has been chosen[26], if no explicit path is given, "mobilenet. tflite" is used by default. It is intended to categorize photographs into 102 different categories and

```

private lateinit var MODEL_PATH:String
private val IMAGE_SIZE = 224
private val NUM_CLASSES = 102

private val REQUEST_IMAGE_CAPTURE = 1
private val REQUEST_IMAGE_GALLERY = 2
private val REQUEST_CAMERA_PERMISSION = 1001

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityFindFlowerBinding.inflate(layoutInflater)
    setContentView(binding.root)

    MODEL_PATH = intent.getStringExtra( name: "ModelPath")?:"mobilenet.tflite"
    val title = intent.getStringExtra( name: "Title")
    binding.btnOpenCamera.setOnClickListener { it: View!
        openCamera()
    }

    binding.btnOpenGallery.setOnClickListener { it: View!
        openGallery()
    }

    binding.btnPredict.setOnClickListener{ it: View!
        val (resultIndexArray,resultMaxArray) = predict()
        if(resultMaxArray[0]<30)
            showTryAgainAlert()
        else {
            val intent = Intent( packageContext: this, ShowResultActivity::class.java)
            intent.putExtra( name: "resultIndexArray", resultIndexArray)
            intent.putExtra( name: "resultMaxArray", resultMaxArray)
            intent.putExtra( name: "Title",title)
            startActivity(intent)
        }
    }
}

```

can be used with images with a size of 224x224 pixels. I added camera and gallery features so that users could take pictures using the device's camera or select ones from the gallery. The activity receives important details, like the model path and a title, when it is first launched from the previous screen. To assess the prediction's accuracy, the outcomes are

Figure 41 Kotlin code for the main page

measured against a predetermined threshold. The user is prompted to try again by an alert if the accuracy drops below a specified threshold (20% in this case). If the prediction turns out to be somewhat accurate, the app moves on to the "ShowResultActivity," where it displays the classification outcomes and the given title. I handled camera permissions through the "REQUEST_CAMERA_PERMISSION"

constant. To deal with situations where the prediction accuracy drops below the predetermined level[26].

I built a function called "showTryAgainAlert()". The invalid message is written inside the function. So this message will pop up if the predictions are less than 20 percent. Also Clicking the "OK" button results in the dialogue box being closed. The classification and prediction of images are handled by the method "predict()". It starts by using the asset manager to access the app's assets. The function then sets the maximum probability for the expected class indices and arrays to store them. If a valid image bitmap is present, the function preprocesses it for inference and resizes it to 224x224, as per the model's recommended size. From the assets, the lite model is loaded into the selected model. After the interpreter has processed the incoming image data, the output array is set up to store classification probabilities. The predicted class index and its corresponding maximum probability are taken from the output array using the arg max function[25]. The corresponding arrays include these results. if there is no suitable image bitmap present, a toast message indicating that there's no valid picture is presented. The function "preprocessImage" is responsible for preparing the input image data in a structure appropriate for the model's inference process. The preprocessed image data are returned in the form of a ByteBuffer by this function after receiving a Bitmap object as input. The input size is initially set to 224x224 pixels and The processed image data is allocated to a ByteBuffer, for compatibility, Using the "getPixels" method, the function extracts the pixel information from the supplied bitmap and stores it in an integer array called "pixels." The red, green, and blue colour channels are divided for each pixel in the "pixels" array and normalized to a range between 0 and 1. These normalized values are then sequentially added as floats to the ByteBuffer. The predicted methods and preprocess image methods are shown in figure 42 and 43 respectively.

```
private fun predict():Pair<ArrayList<Int>,ArrayList<Int>> {
    val assetManager = assets
    var predictedClassIndexArray = arrayListOf<Int>()
    var predictedClassMaxArray = arrayListOf<Int>()
    if (imageBitmap != null) {
        val resizedBitmap = Bitmap.createScaledBitmap(imageBitmap!!, dstWidth: 224, dstHeight: 224, filter: true)
        val inputValues = preprocessImage(resizedBitmap)
        val tflite = Interpreter(loadModelFileFromAsset(assetManager, MODEL_PATH))
        val output = Array<size: 1> { FloatArray(NUM_CLASSES) }
        tflite.run(inputValues, output)
        val result = argmax((output[0]))
        predictedClassIndexArray = result.first
        predictedClassMaxArray = result.second
    } else {
        Toast.makeText(context: this, text: "Invalid image", Toast.LENGTH_SHORT).show()
    }
    return Pair(predictedClassIndexArray, predictedClassMaxArray)
}
```

Figure 42 Kotlin code for predict function

```
private fun preprocessImage(bitmap: Bitmap): ByteBuffer {
    val inputSize = 224
    val output = ByteBuffer.allocateDirect(capacity: 4 * inputSize * inputSize * 3).apply { this: ByteBuffer
        order(ByteOrder.nativeOrder())
    }

    val pixels = IntArray(size: inputSize * inputSize)
    bitmap.getPixels(pixels, offset: 0, inputSize, x: 0, y: 0, inputSize, inputSize)

    for (pixel in pixels) {
        val r = (pixel shr 16 and 0xFF) / 255.0f
        val g = (pixel shr 8 and 0xFF) / 255.0f
        val b = (pixel and 0xFF) / 255.0f

        output.putFloat(r)
        output.putFloat(g)
        output.putFloat(b)
    }
}
```

Figure 43 Kotlin code for preprocess image function

The binary data for the model is retrieved from the app's assets by the "load Model File From Asset" function. To accomplish this, it first opens the model file in the asset management, gathers the appropriate file data, and then creates a read-only memory-mapped Byte Buffer. The model's data are contained in this ByteBuffer [26], to enable inference. I added a method to the "argmax" function that allows me to find the indices and values of the top three entries in a specified FloatArray. This procedure helps determine which class predictions are more likely based on the output probabilities of the model. In order to do this, I set up variables called firstMax, secondMax, and thirdMax to keep track of the three highest values and the indices that go with them. To access both the index and value of each element, I iterated through the input array using the "arr.withIndex()" . I updated the variables during the iteration by comparing each value to the current maximum values. When a new maximum

```
fun argmax(arr: FloatArray): Pair<ArrayList<Int>, ArrayList<Int>> {
    var firstMax = Float.NEGATIVE_INFINITY
    var secondMax = Float.NEGATIVE_INFINITY
    var thirdMax = Float.NEGATIVE_INFINITY

    var firstIndex = -1
    var secondIndex = -1
    var thirdIndex = -1

    for ((index, num) in arr.withIndex()) {
        if (num > firstMax) {
            thirdMax = secondMax
            thirdIndex = secondIndex
            secondMax = firstMax
            secondIndex = firstIndex
            firstMax = num
            firstIndex = index
        } else if (num > secondMax) {
            thirdMax = secondMax
            thirdIndex = secondIndex
            secondMax = num
            secondIndex = index
        } else if (num > thirdMax) {
            thirdMax = num
            thirdIndex = index
        }
    }

    val indexArray = arrayListOf<Int>(firstIndex, secondIndex, thirdIndex)
    val maxArray = arrayListOf<Int>((firstMax*100).toInt(), (secondMax*100).toInt(), (thirdMax*100).toInt())
    return Pair(indexArray, maxArray)
}
```

was found, I adjusted the prior maximum numbers to accurately monitor the top three. Once everything was put together, I created two arrays: "indexArray," which contains the indices of the top three elements, and "maxArray," which contains the % probability for each of those probabilities. These arrays offer insightful information about the most likely class prediction. The below figure 45 shows the Kotlin code of how I implemented the results page activity which is the last page. In the

Figure 44 Kotlin code for finding top three prediction

showresults activity, I created the functionality to show the predictions from our model for flower recognition. In order to achieve this, activity show results binding class is employed which allows to binding of the layout components. The predicted flower names, corresponding percentages, and brief descriptions are displayed in this activity, which I created by incorporating various crucial elements and concepts. Also, I have obtained these data from the previous activity. There are three arrays that will have flower title, flower prediction list, and their respective flower description. If second predictions have a prediction percentage higher than 10%, they are also added to these arrays along with the first highest flower prediction details.


```

class ShowResultActivity : AppCompatActivity() {
    lateinit var binding: ActivityShowResultBinding
    lateinit var resultIndexArray : ArrayList<Int>
    lateinit var resultMaxArray : ArrayList<Int>
    val flowersList= arrayListOf<String>("Pink Primrose","Globe Thistle","Blanket Flower","True Love")
    val flowerDesc = arrayListOf<String>("Pink Primrose (Primula vulgaris): Pink primrose is a small, daisy-like flower with pink petals and a yellow center.", "Globe Thistle (Echinops globosus): A tall, slender flower with a globe-shaped head of small, blue-purple flowers.", "Blanket Flower (Gaillardia): A flower with a flat, daisy-like head of small, yellow, orange, and red flowers.", "True Love (Anemone): A flower with a flat, daisy-like head of small, white, pink, and red flowers.")
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityShowResultBinding.inflate(layoutInflater)
        setContentView(binding.root)
        resultIndexArray = intent.getIntegerArrayListExtra( name: "resultIndexArray") as ArrayList<Int>
        resultMaxArray = intent.getIntegerArrayListExtra( name: "resultMaxArray") as ArrayList<Int>
        binding.textView.text = intent.getStringExtra( name: "Title")+" Model Prediction Results"
        var flowerTitleList = ArrayList<String>()
        var flowerPercentageList = ArrayList<String>()
        var flowerDescList = ArrayList<String>()
        flowerTitleList.add(flowersList[resultIndexArray[0]])
        flowerPercentageList.add(resultMaxArray[0].toString()+"%")
        flowerDescList.add(flowerDesc[resultIndexArray[0]])
        if(resultMaxArray[1]>10) {
            flowerTitleList.add(flowersList[resultIndexArray[1]])
            flowerPercentageList.add(resultMaxArray[1].toString()+"%")
            flowerDescList.add(flowerDesc[resultIndexArray[1]])
        }
        if(resultMaxArray[2]>10) {
            flowerTitleList.add(flowersList[resultIndexArray[2]])
            flowerPercentageList.add(resultMaxArray[2].toString()+"%")
            flowerDescList.add(flowerDesc[resultIndexArray[2]])
        }

        val recyclerView = findViewById<RecyclerView>(R.id.recyclerView)
        val adapter = ResultListAdapter(flowerTitleList,flowerPercentageList,flowerDescList)
        recyclerView.setHasFixedSize(true)
        recyclerView.layoutManager = LinearLayoutManager( context: this)
        recyclerView.adapter = adapter
    }
}

```

Figure 45 Kotlin code for predicting results

I have decided to give a flower description of the predicted flower as an additional feature. So that the user will get to know more about the flower. Giving a description directly in the results screen makes the app work even without an internet connection rather they include the Wikipedia link. On the other hand, many flowers have no Wikipedia. this may lead to show error or blank screen when I import Wikipedia to show more information about flowers. In this chapter, I have explained important functions that I used.

6. Functionality Testing

Once I had successfully integrated it I started to use my app by testing it. Since this has two models, the prediction will vary of the same image by trying with two models. Figure 46 and 47 shows the prediction of the image I uploaded to test my app in the mobile net model as well as the VGG model. The actual name of this flower is fire lily. The mobile net model predicts the flower correctly with 100 % accuracy which shows the potential of my app later when I tested the same image with VGG it predicted correctly with a confidence of 97%. This is because my VGG model has less accuracy.

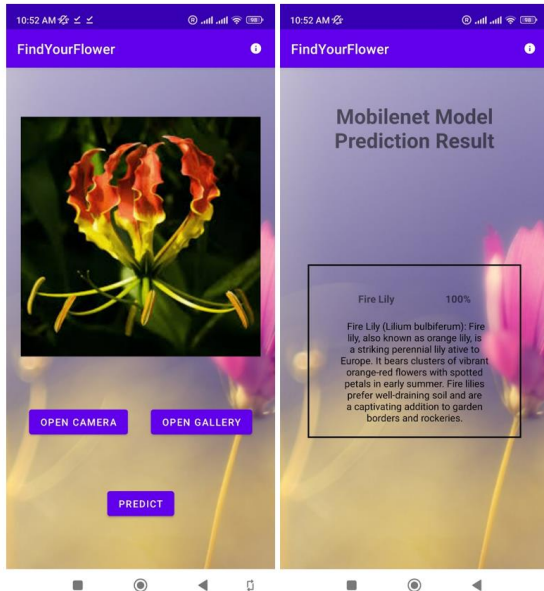


Figure 46 Firelily prediction using mobile net

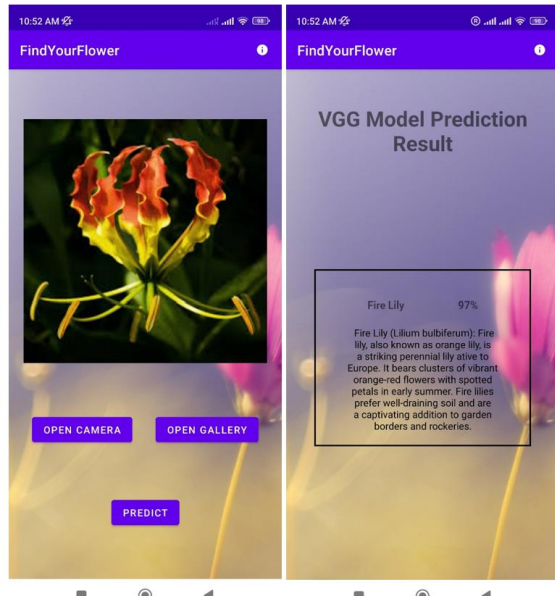


Figure 47 Firelily prediction using vgg16

Figure 48 and 49 show another example of the prediction of my app. I uploaded the image of the water lily using the mobile net model. when I click the predict button It will show the results on the next screen. The results are water lily which is a correct prediction. the confidence in this prediction is 99%. But with the VGG model, it showed 2 predictions. the water lily with 86% and lotus with 13%. The lotus is a wrong prediction but the reason why the app predicted it as lotus is water lily and lotus looks the same. So the model showed all the prediction possibilities.

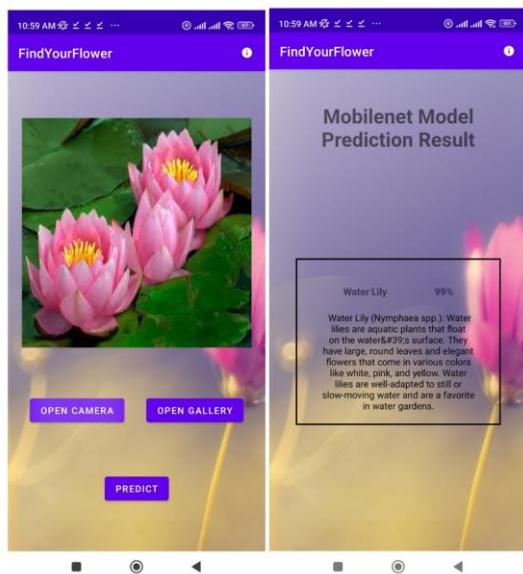


Figure 48 Lotus prediction by MobileNet

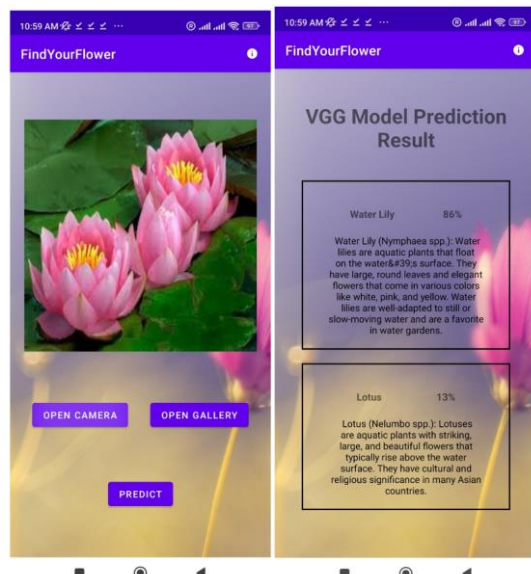
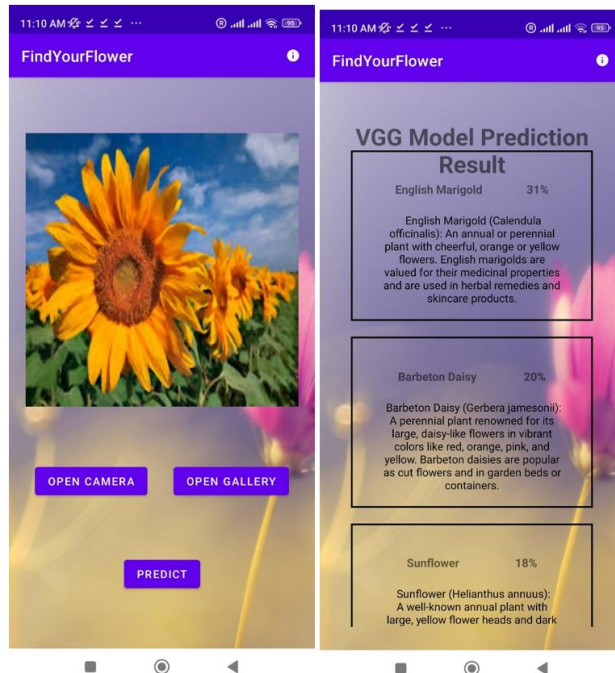


Figure 49 Lotus prediction by VGG16

Figure 50 shows the wrong prediction of the flower images. The image passed into the mobile app is



The image passed into the mobile app is sunflower. The VGG model predicts it as wrong in the first prediction. It is showing as marigold which is wrong. The second prediction is barbeton daisy which is also wrong. It also shows the third prediction as their confidence was more than 10 %. We should notice that the model is able to predict the right name of the flower at the third one. Which means that there is a high chance that flower name may be predicted as right in the top three predictions.

In order to accurately evaluate the effectiveness of the testing features in my app, I decided to compare it with a well-established app available on the Play Store, below figure 51 and 52 shows the comparison of my app against the Pl@ntnet app. PlantNet may recognise different plant species with the help of images user take, when I uploaded

Figure 50 Sunflower prediction using VGG16

a photo of *cautleya spicata* my model predicted the flower correctly with a confidence of 99%. Whereas the pl@ntnet app is predicting the flower with a prediction of 77%. This shows that findyourflower is performing better than Pl@ntnet. pl@ntnet has many features other than predicting like proving information and a interface of the app is built with high level of complexity. It also shows the results of similar photos of the same category.

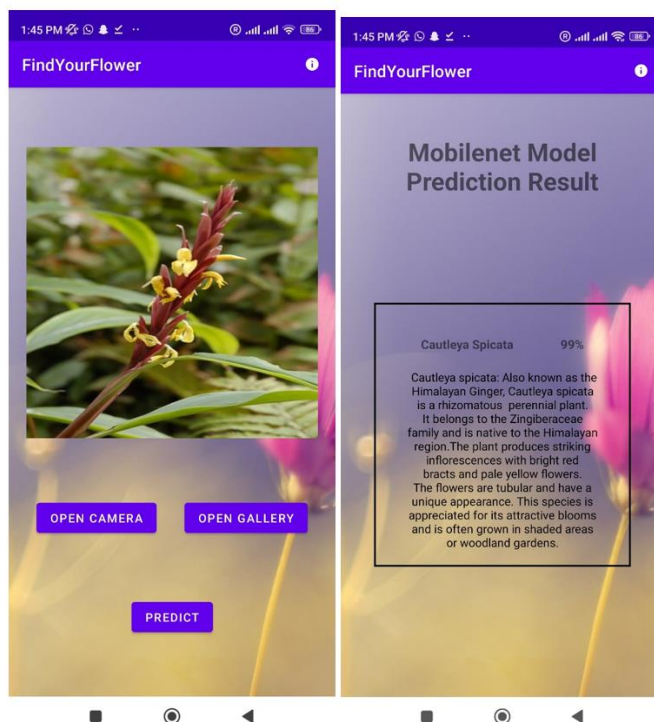


Figure 51 Mobilenet prediction of *cautleya spicata*

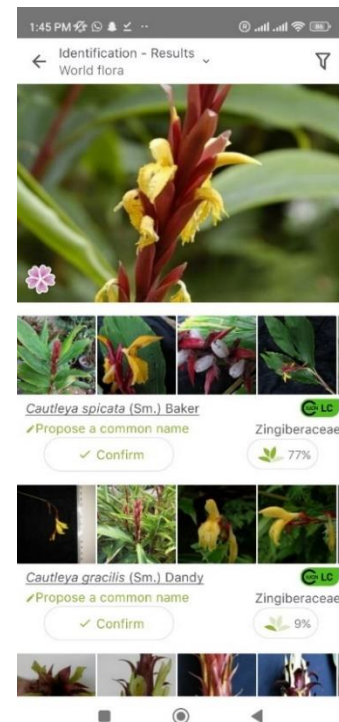


Figure 52 Pl@ntnet prediction

Seek by inaturalist [28] is the another app I found in playstore which predicts plant, flowers, and other species. I also compared the findyourflower prediction with this. It is found that seek is having very good design structure of the app but the predictions are quite low. the app could not identify the flower images properly. I tested by passing the image of a common dandelion, seek shows that the uploaded flower can be a member of any below family rather than showing the exact prediction. This means that this app couldn't recognize images properly. Apparently, findyourflower recognizes the images as common dandelion with 87% of confidence which is correct prediction. It also shows another flower name with a prediction of 12 percentage in Figure 54.

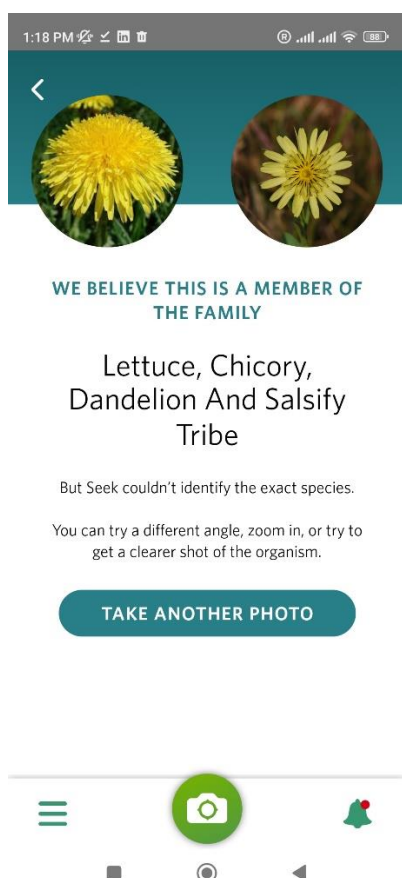


Figure 53 Seek by Inaturalist prediction

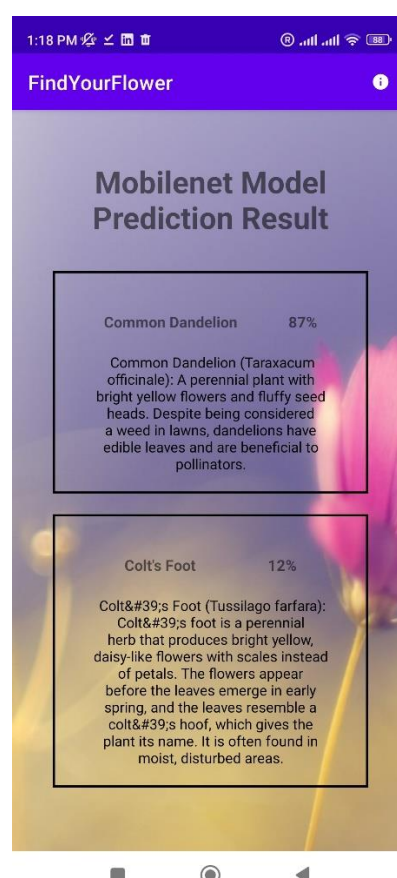


Figure 54 Common dandelion prediction

I also tested FindYourFlower by clicking random pictures of flowers that I found on the streets and in university. Figure 55 shows a picture of rose .FindYourFlower predicts the images correctly as rose. This is by using mobilenet model. Figure 56 shows the picture of another rose which is yellow. My app predicts this image as rose with confidence 64 percentage and camelia with 22 %. This means that my model is predicting same species in different colours. Figure 57 shows the images of a white flower that I found near university. FindYourFlower is identifying this as primula with 61% and garden Phlox with 34%.

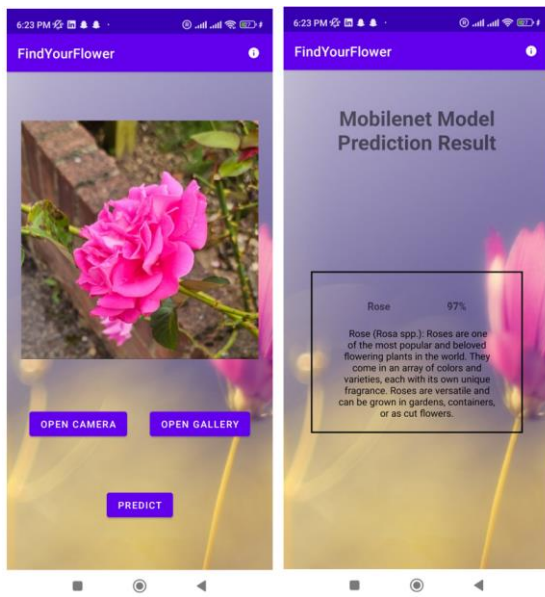


Figure 55 Prediction of pink rose

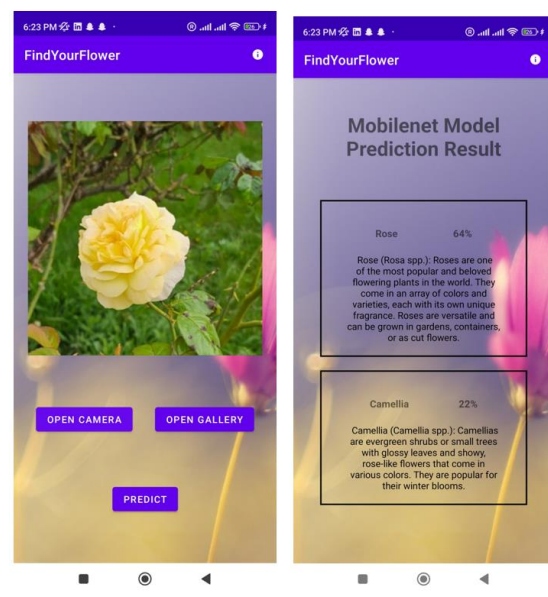


Figure 56 Prediction of yellow Rose

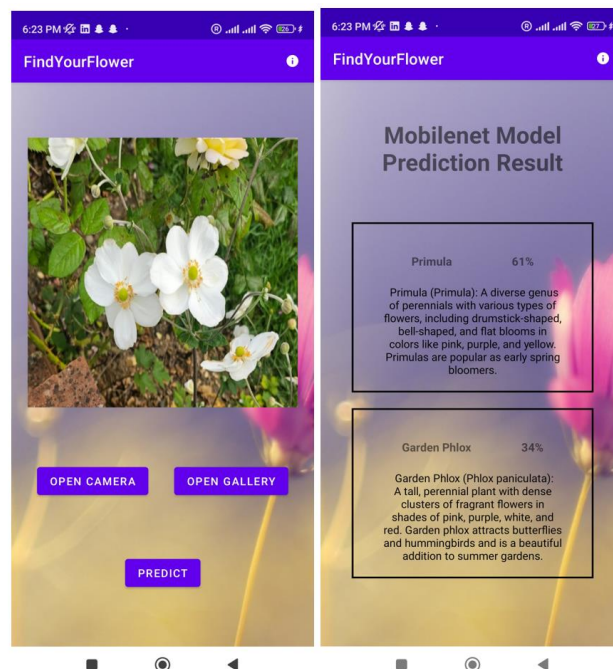


Figure 57 Prediction of White Flower

7. User Feedback Analysis

After I completed my app and converted it into an APK file so that Android devices could easily install it. Upon discussing with my supervisor, it was suggested that I gather feedback from potential users as a part of the user testing phase. I created a Google Form[29] with specific questions on the functionality, usability, and features of the application in order to facilitate the feedback collection process. After finalizing the Google Form, I shared the APK file with users. I provide full instructions on how to set up and use the application along with the APK file. I also asked them to complete the Google Form whenever they got a chance to use the app. This form[29] helps me in collecting opinions on various aspects of the application's user experience.

The first question is based on how accurate the flower predictions are when the user takes a picture and uploads it. For that, the responses are like 50 % scored 4 33 % scored 5, and 16 % scored 3. Based on the distribution of scores received for flower predictions, it is evident that the app's performance falls within the moderate prediction accuracy range.

How accurate were the predictions you received?

6 responses

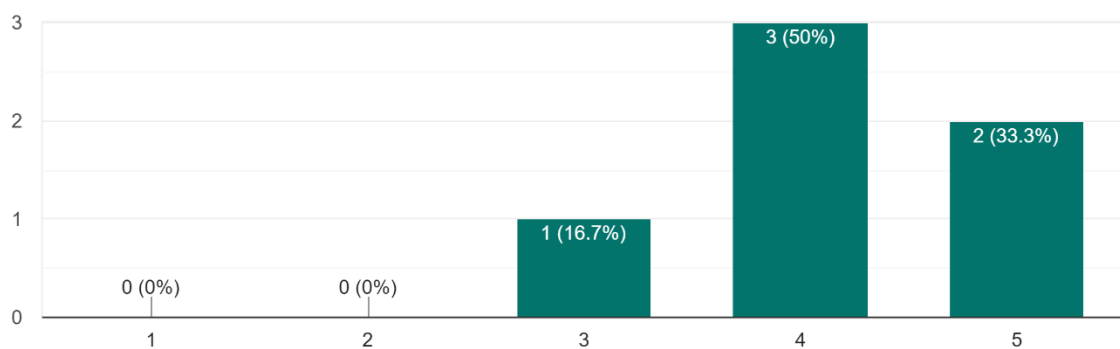


Figure 58 Prediction User Feedback

One of the primary aspects of reviewing the application is how the interface of the app has been done. The feedback in Figure 59 showed that the users had responded quite favourably. 50% of people who responded gave a rating of 4, while the remaining 50% gave a maximum rating of 5. The distribution of scores shows that the interface design is generally viewed favourably. The balanced scores between 4 and 5 demonstrate that the majority of users consider the interface to be simple to use, visually pleasing, and functionally useful in making it easier to recognize flowers. This suggests that the design met their expectations and provided a hassle-free experience.

How was the interface of app

6 responses

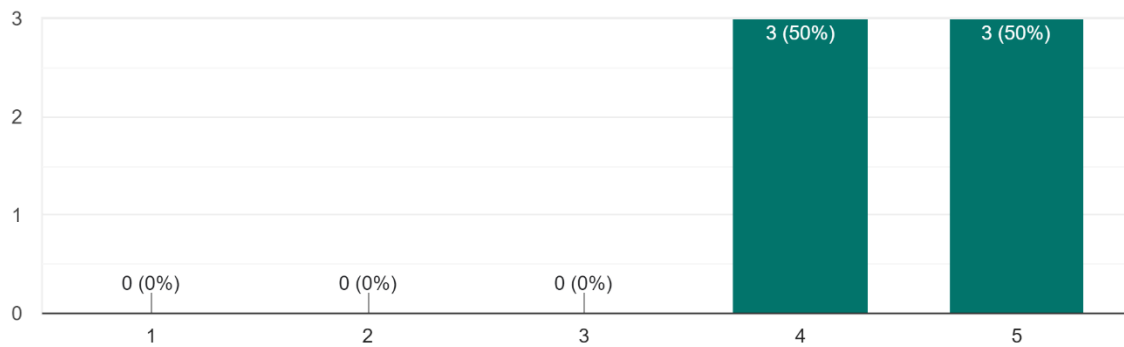


Figure 59 User feedback on the interface

The evaluation of the app's performance is also assessed through user feedback. The below feedback data shows that the performance of the app was very well received. A significant majority of 66% of the participants gave the highest rating of 5, while 33% gave a rating of 4. The prevalence of scores 5 and 4 shows users have consistently had consistent and good outcomes in terms of accuracy and user experience without facing any bug issues.

Rate the performance of the app

6 responses

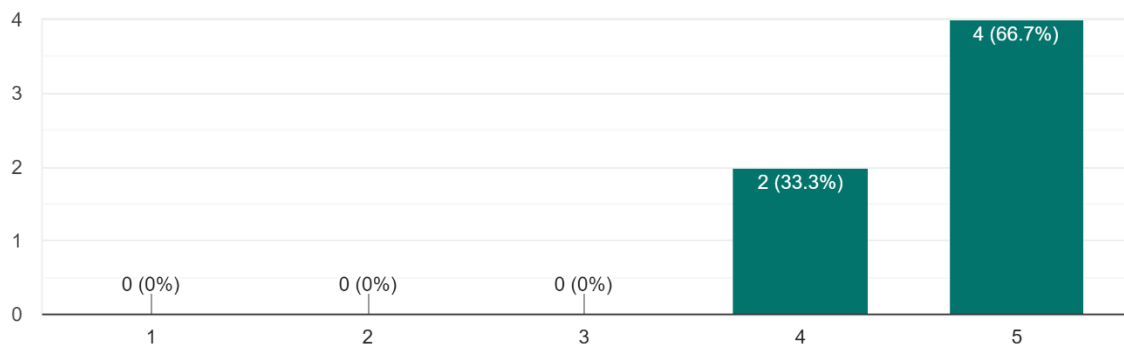


Figure 60 Performance user feedback

Another question I have included in the form is to get feedback on what prediction the user has made. From the feedback in Figure 61, one user gave that one of the predictions was wrong which is due to 87% of the model accuracy. The rest of the users mentioned that the predictions were almost correct. While recognizing the overall success in providing accurate prediction, it draws attention to the need to constantly enhance model accuracy.

please give feedback on predictions you made

6 responses

Predictions are accurate as pr my knowledge
I tried testing with dandelion,The accuracy is 98% which is good.
captured image using this application and it is giving good predictions
Predictions are correct
Tired capturing flowers, one was wrong

Figure 61 Feedback on predictions made

On a scale of 1 to 10, users' opinions on how satisfied they are with the app's current status are rated. It is clear from the Figure 62 data that the distribution of scores falls into a specific pattern: 33% of users gave a score of 10, while the remaining 66% gave a rating of 9. This shows that the users are highly satisfied with the app's features, performance, and overall usability.

On a scale of 1 to 10, how satisfied are you with the app's current state?

6 responses

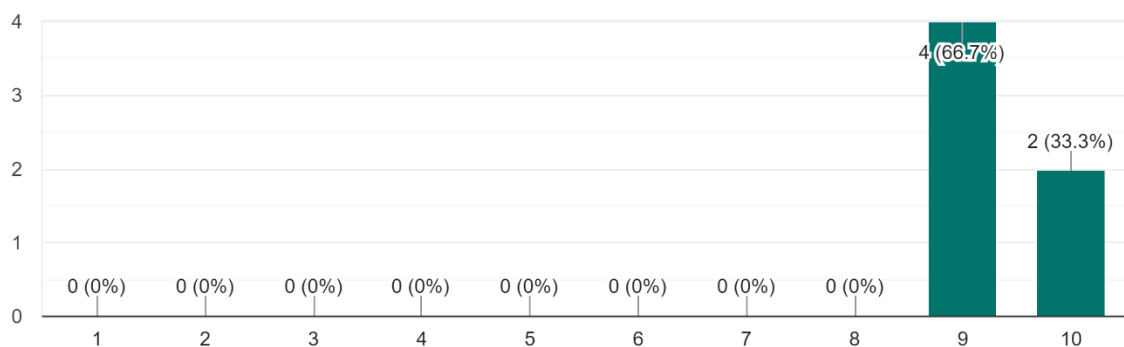


Figure 62 Overall rating feedback

8. Project Management

8.1 Project Plan

The aim is to develop not only a successfully running mobile application but also a model with good accuracy that can predict flowers correctly. To start with this, I produced a Gantt chart where I set deadlines for all the processes required for this project starting from background research. The project plan was driven using Gantt charts, which enable the systematic execution of tasks, ensuring a balanced approach to both model development and app creation. I also used two project management tools like Jira and GitLab which are provided by the School of Computer Science and Electronic Engineering.

8.2 Use of Jira

Below figure 63 shows the kanban board of module CE901. Throughout the project's duration, I consistently utilized Jira software as a crucial tool for organizing and transparently communicating the status of my work. The method I employed involved the creation of user stories and issues on a weekly basis, immediately following my meetings with my project manager. This strategy worked incredibly well at giving me a clear roadmap for each week, which helped me prioritize and concentrate on the tasks at hand.

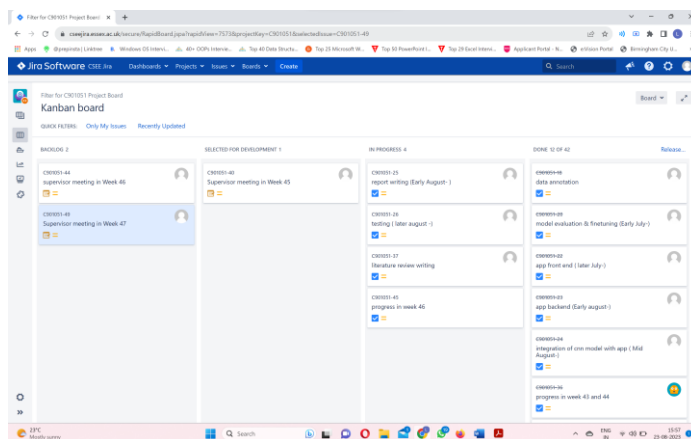


Figure 63 Jira dashboard

The below Figure 64 shows the Jira cumulative diagram which shows the progress done during the project periods. This includes creating tasks. most of the stories were created at the start of June when I started working on the project. At the beginning of march, there are a few stories created and has been done as a part of the project proposal. All the tasks will be moved to done once its been finished. There are about 50 stories created throughout the entire work.

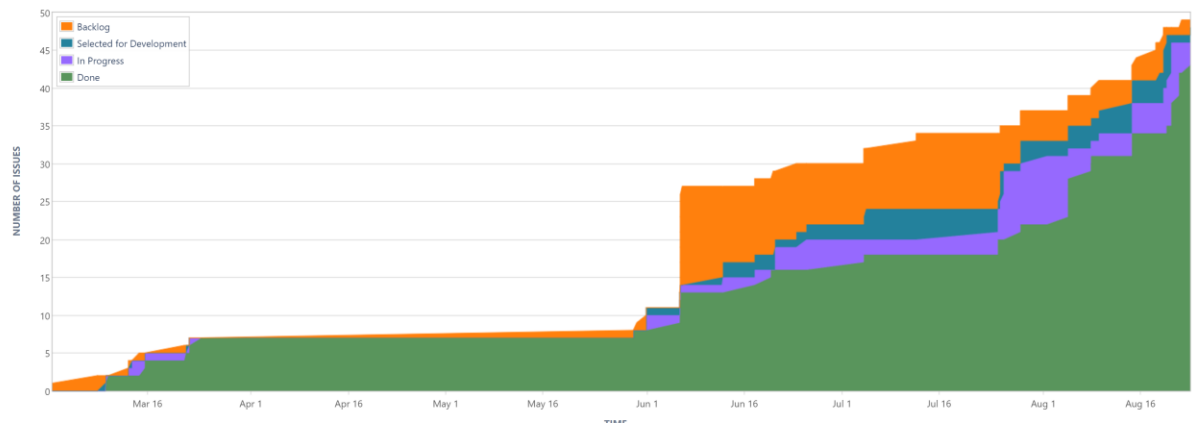


Figure 64 Cumulative flow diagram of Jira

The below Figure 65 shows the created vs. resolved issues graph. Initially there was a constant balance between the created and solved. Then there were no issues created in April when examination was conducted May which is a summer break. At the beginning of June, there were more issues created. This is due to my professor's suggestion to create all tasks initially. This is understandable given that I had several stories and issues set forward during the initial development stage that needed to be resolved. So I started creating all the stories after each meetings for the entire week. On the other hand, the issues were resolved once it's done solved.

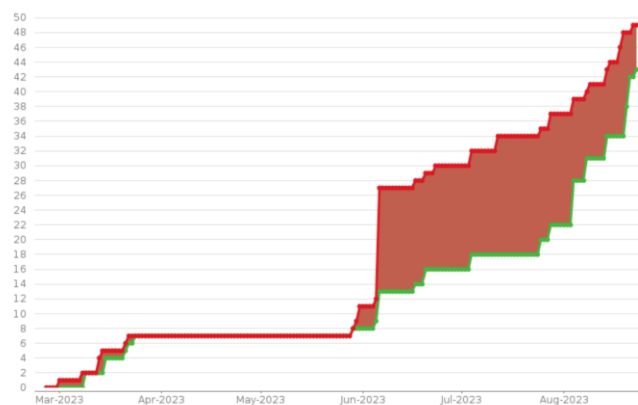


Figure 65 Created Vs Resolved graph

8.3 Use of GitLab

In my project, I effectively utilized GitLab as a robust version control tool to protect my code and report. I made sure there was a solid backup system and a thorough history of modifications by continuously committing new code and models as soon as they were stable. When working across several machines, this strategy was very helpful because it allowed me to easily clone the repository and continue working without interruption. During the duration of the entire project, GitLab was essential in keeping things organized, collaborative, and continuous.

9. Conclusion

In conclusion, The development of the "FindYourFlower" app for mobile devices represents a significant achievement in classifying flowers using deep learning. I have been able to train a model with reasonable accuracy as this was my the project's main purpose. Along with that, I have also trained another model to recognize flowers using the same Oxford 102 flower dataset. The final mobile net model can achieve an accuracy of approximately 87% which gives good results through actual vs. predicted classes. The successful completion of the project shows how deep learning techniques can be used in everyday life, especially in the area of image recognition. The trained mobile net model is incorporated into the app to classify images. Though the accuracy of VGG16 is low as compared with mobile net, it is also integrated into the app which is a plus. The primary intent of the project was to develop an effective and user-friendly device for anyone interested in flower identification. This goal is achieved with the "FindYourFlower" app, which has a user-friendly interface that enables users to take or provide pictures of flowers instantly and receive predictions that are accurate. The app works well because it uses both MobileNet and VGG models. This makes it very good at recognizing many different kinds of flowers. This states that I have successfully developed a mobile app that classifies the flower with high accuracy. However, I have also learned about deep learning models and how to integrate them into a developed mobile application. Several difficulties were faced and resolved during the development process. Every step of the process, from data preprocessing through model training and integration into the app's architecture, needed more time. In the era of deep learning, FindYourFlower illustrates its practical application.

9.1 Future Enhancement

There are several limitations in the trained model and also in the android app. These limitations can be improved as a part of future enhancement

A. Model Improvement

As mentioned in the model evaluation, my model is facing slight overfitting. This should be improved in future work.

I can also get good results with other models like Efficientnet or Resnet. The results of the model might be improved by using different models.

Although my model does not give 100% accuracy. There are many images to which my app prediction is wrong. This is due to the dataset it was trained on. So I can improve my model by using the larger dataset. And also the images in the dataset should have a variety of angles and positions.

B. App Improvement

The current version of Findyourflower can be improved in many ways. The UI of the current version is very simple so this can be improved by adding some additional information or design.

When I rotate my mobile to portrait this app enables its features to rotate in this direction. However, the functionality of the app is stopping. This should be resolved in the future.

Initially, I thought of adding the image of 102 flowers and their names correspondingly in a new screen as a part of extra features. This will be useful for any user to know the image of an actual flower.

The size of the app is around 120 MB. This can be reduced by using backend database. The ML models can be loaded through ML kit in Firebase which has the advantage of giving faster results while reducing the size of the implementation.

Bibliography

- [1] Sharifani, K. and Amini, M., 2023. Machine Learning and Deep Learning: A Review of Methods and Applications. *World Information Technology and Engineering Journal*, 10(07), pp.3897-3904.
- [2] www.mathworks.com. (n.d.). *What Is Image Recognition?* [online] Available at: <https://www.mathworks.com/discovery/image-recognition-matlab.html#:~:text=Image%20Recognition%20Using%20Machine%20Learning> [Accessed 06 Aug. 2023].
- [3] Murray, R.F., 2011. Classification images: A review. *Journal of vision*, 11(5), pp.2-2
- [4] Jones, H.G. (2020). What plant is that? Tests of automated image recognition apps for plant identification on plants from the British flora. *AoB PLANTS*, 12(6). doi:<https://doi.org/10.1093/aobpla/plaa052>.
- [5] Narvekar, C. and Rao, M., 2020, December. Flower classification using CNN and transfer learning in CNN-Agriculture Perspective. In *2020 3rd international conference on intelligent sustainable systems (ICISS)* (pp. 660-664). IEEE.
- [6] Gavai, N.R., Jakhade, Y.A., Tribhuvan, S.A. and Bhattad, R., 2017, December. MobileNets for flower classification using TensorFlow. In *2017 international conference on big data, IoT and data science (BID)* (pp. 154-158). IEEE.
- [7] Sinha, D. and El-Sharkawy, M., 2019, October. Thin mobilenet: An enhanced mobilenet architecture. In *2019 IEEE 10th annual ubiquitous computing, electronics & mobile communication conference (UEMCON)* (pp. 0280-0285). IEEE.
- [8] Arfan, T.H., Hayaty, M. and Hadinegoro, A., 2021, September. Classification of brain tumours types based on MRI images using mobilenet. In *2021 2nd International Conference on Innovative and Creative Information Technology (ICITech)* (pp. 69-73). IEEE.
- [9] Abas, M.A.H., Ismail, N., Yassin, A.I.M. and Taib, M.N., 2018. VGG16 for plant image classification with transfer learning and data augmentation. *International Journal of Engineering and Technology (UAE)*, 7(4), pp.90-94.
- [10] Ardito, L., Coppola, R., Malnati, G. and Torchiano, M., 2020. Effectiveness of Kotlin vs. Java in android app development tasks. *Information and Software Technology*, 127, p.106374.
- [11] Back Garden - Gardening Blog & Guides for Gardeners. (2020). *9 Best Free Plant Identification Apps For Android & iOS (2021)*. [online] Available at: <https://backgarden.org/plant-identification-apps/>.

- [12] Pl@ntNet (2022). *Pl@ntNet offline/embedded: identify plants anywhere without connection*. [online] Pl@ntNet. Available at: <https://plantnet.org/en/2022/10/18/plntnet-offline-embedded-identify-plants-anywhere-without-connection/> [Accessed 20 Aug. 2023].
- [13] www.kaggle.com. (n.d.). *Flowers Recognition*. [online] Available at: <https://www.kaggle.com/datasets/alxmamaev/flowers-recognition>.
- [14] www.robots.ox.ac.uk. (n.d.). *Visual Geometry Group - University of Oxford*. [online] Available at: <https://www.robots.ox.ac.uk/~vgg/data/flowers/102/>.
- [15] Jmour, N., Zayen, S. and Abdelkrim, A., 2018, March. Convolutional neural networks for image classification. In *2018 international conference on advanced systems and electric technologies (IC_ASET)* (pp. 397-402). IEEE.
- [16] Shaha, M. and Pawar, M., 2018, March. Transfer learning for image classification. In *2018 second international conference on electronics, communication and aerospace technology (ICECA)* (pp. 656-660). IEEE.
- [17] Ertam, F. and Aydın, G., 2017, October. Data classification with deep learning using Tensorflow. In *2017 international conference on computer science and engineering (UBMK)* (pp. 755-758). IEEE.
- [18] TensorFlow. (2019). *Keras | TensorFlow Core | TensorFlow*. [online] Available at: <https://www.tensorflow.org/guide/keras>.
- [19] Yadav, H. (2022). *Dropout in Neural Networks*. [online] Medium. Available at: <https://towardsdatascience.com/dropout-in-neural-networks-47a162d621d9>.
- [20] Singh, P. and Manure, A., 2019. *Learn TensorFlow 2.0: Implement Machine Learning and Deep Learning Models with Python*. Apress.
- [21] TensorFlow. (n.d.). *tf.keras.applications.mobilenet.MobileNet | TensorFlow Core v2.8.0*. [online] Available at: https://www.tensorflow.org/api_docs/python/tf/keras/applications/mobilenet/MobileNet.
- [22] TensorFlow. (n.d.). *tf.keras.applications.vgg16.VGG16 | TensorFlow v2.13.0*. [online] Available at: https://www.tensorflow.org/api_docs/python/tf/keras/applications/vgg16/VGG16 [Accessed 20 Jun. 2023].
- [23] Hanafiah, M., Adnan, M.A., Abdul-Rahman, S., Mutalib, S., Ab Malik, A.M. and Shamsuddin, M.R., 2022, April. Flower recognition using deep convolutional neural networks. In *IOP Conference Series: Earth and Environmental Science* (Vol. 1019, No. 1, p. 012021). IOP Publishing.

- [24] Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A. and Zou, J., 2019. Gradio: Hassle-free sharing and testing of ml models in the wild. *arXiv preprint arXiv:1906.02569*.
- [25] Sehgal, A. and Kehtarnavaz, N., 2019. Guidelines and benchmarks for deployment of deep learning models on smartphones as real-time apps. *Machine Learning and Knowledge Extraction*, 1(1), pp.450-465.
- [26] Moskala, M. and Wojda, I., 2017. *Android Development with Kotlin*. Packt Publishing Ltd.
- [27] Karthikeyan, N.G., 2018. *Machine Learning Projects for Mobile Applications: Build Android and IOS Applications Using TensorFlow Lite and Core ML*. Packt Publishing Ltd.
- [28] play.google.com. (n.d.). *Seek by iNaturalist - Apps on Google Play*. [online] Available at: <https://play.google.com/store/apps/details?id=org.inaturalist.seek&hl=en&gl=US> [Accessed 01 Aug. 2023].
- [29] Google Docs. (n.d.). *User Feedback*. [online] Available at: <https://docs.google.com/forms/d/17hh0mM0URtCCSzJFLDbx4FSirhWp4T20QyibGeZsoWY/edit> [Accessed 15 Aug. 2023].

Appendices

Source code of this project is available in my Gitlab Repository. The link for accessing my repository is https://cseegit.essex.ac.uk/22-23-ce901-ce902-su/22-23_CE901-CE902-SU_bodhanapu_lakshmipriya

Figure 66 shows the folders of my work throughout the dissertation. Dataset folder will have all the datasets I used for training model. Final_dissertation Folder will have a subfolder named Implementation and a copy of this report. The source of the app and model are given in the Implementation folder. I have also provided a readme file that explains how to run the code.

Name	Last commit
app	commit
dataset	Upload New File
final model	Upload New File
final_dissertation/Implementation	tflite model uploaded
model	Upload New File
report	app development

Figure 66 Source code directory

How to run:

- The model code is in python language that can be implemented either Jupyter or Google Colab. Colab is preferred for faster GPU.
- The Android code can be implemented using Android Studio.
- Installation of Java, Python and Android Studio are mandatory.